TAGCAN

Generated by Doxygen 1.9.8

1 Documentation TAGCAN 1
1.1 Contenu . . . 1
1.2 Perimetre . . . 1

2 API Reference (Etat actuel) 3
2.1 Vue d'ensemble . . . 3
2.2 Modules . . . 3
2.3 Structures et etat global . . . 3
2.3.1 <tt>TagDef</tt> (<tt>include/sub_include/tag_manager.h</tt>) . . . 3
2.3.2 Variables globales . . . 4
2.4 Fonctions principales . . . 4
2.4.1 Gestion des tags . . . 4
2.4.2 Valeurs (payload) . . . 4
2.5 Debug / affichage . . . 5
2.6 Codes de retour . . . 5
2.6.1 Enum <tt>CAN_TG_STATUE</tt> (<tt>include/config_value.h</tt>) . . . 5
2.6.2 Constantes globales (<tt>scripts/config_value.c</tt>) . . . 5
2.7 Notes importantes de comportement . . . 5

3 01 - Initialisation de TAGCAN 7
3.1 Objectif . . . 7
3.2 Vue d'ensemble du processus . . . 7
3.3 Détail de chaque étape . . . 7
3.3.1 Étape 1 : init_data_container() . . . 7
3.4 - Atention: un appel en cours d'exécution réinitialiseras les valeurs à 0. . . . 8
3.4.1 Étape 2 : init_tag_manager() . . . 8
3.4.2 Étape 3 : CAN_setup_TAG() . . . 8
3.4.3 Étape 4 : CAN_setup_DF() . . . 8
3.4.4 Étape 5 : Activer les interruptions . . . 8
3.5 Code complet d'initialisation . . . 9
3.6 Points d'attention . . . 9
3.7 Vérification . . . 9

4 02 - Gestion des Tags 11
4.1 Qu'est-ce qu'un tag ? . . . 11
4.2 Créer un tag : set_tag() . . . 11
4.2.1 Prototype . . . 11
4.2.2 Paramètres . . . 11
4.2.3 Valeur de retour . . . 11
4.2.4 Exemples . . . 11
4.3 Écrire une valeur : set_value() . . . 11
4.3.1 Prototype . . . 11
4.3.2 Paramètres . . . 11

4.3.3 Valeur de retour . . . 12
4.3.4 Comportement important . . . 12
4.3.5 Exemples d'utilisation . . . 12
4.4 Lire une valeur : get_value() . . . 12
4.4.1 Prototype . . . 12
4.4.2 Paramètres . . . 12
4.4.3 Valeur de retour . . . 12
4.4.4 Exemples . . . 13
4.5 Workflow complet : set / get / send . . . 13
4.5.1 Scénario d'exemple . . . 13
4.6 Bonnes pratiques . . . 14
4.7 Schéma de tags recommandé . . . 14

5 03 - Manipulation du Buffer de Données CAN **15**
5.1 Objectif . . . 15
5.2 Structure du buffer CAN . . . 15
5.2.1 Taille et format . . . 15
5.2.2 Hiérarchie . . . 15
5.3 Fonctions de gestion du buffer . . . 15
5.3.1 1 init_data_container() . . . 15
5.3.2 2 set_val_TxData() . . . 15
5.3.3 3 can_simulate_send_receive() . . . 16
5.4 Workflow complet : Écriture -> Envoi -> Réception . . . 16
5.4.1 Émission (Tx) . . . 16
5.4.2 Code d'émission . . . 16
5.4.3 Réception (Rx) . . . 17
5.4.4 Code de réception . . . 17
5.5 Directement lire/écrire les octets . . . 17
5.5.1 Lecture directe . . . 17
5.5.2 Écriture directe . . . 17
5.6 Pièges courants . . . 17
5.7 Tableau d'aide : Qui fait quoi ? . . . 18
5.8 Exemple : Envoi -> Réception -> Traitement . . . 18

6 04 - Filtrage CAN **21**
6.1 Objectif . . . 21
6.2 Contexte . . . 21
6.3 Fonction de filtrage . . . 21
6.3.1 Prototype . . . 21
6.3.2 Paramètres . . . 21
6.3.3 Valeur de retour . . . 21
6.4 Comment ça marche ? . . . 21
6.4.1 Le filtre masque (IDMASK) . . . 21

6.5 Exemple de configuration . . . 22
6.5.1 Cas 1 : Accepter une plage d'IDs (0x100 à 0x120) . . . 22
6.5.2 Cas 2 : Accepter un ID exact (0x123) . . . 22
6.5.3 Cas 3 : Accepter tous les messages . . . 22
6.6 Détails de la configuration interne . . . 22
6.7 Exemple d'intégration complet . . . 22
6.7.1 Initialisation avec filtrage . . . 22
6.8 Limitation des bancs de filtrage . . . 23
6.9 Stratégies de filtrage recommandées . . . 23
6.9.1 Stratégie 1 : Filtrage par catégorie (Recommandé) . . . 23
6.9.2 Stratégie 2 : Un filtre par source . . . 24
6.10 Points d'attention . . . 24
6.11 Débuggage des filtres . . . 24
6.12 Comparaison : Avec/Sans filtres . . . 24
6.12.1 Sans filtres (accepte tout) . . . 24
6.12.2 Avec filtres (recommandé) . . . 25

7 05 - Exemples Complets **27**
7.1 Exemples d'intégration pratiques . . . 27
7.2 Exemple 1 : Système de télémétrie simple . . . 27
7.2.1 Scénario . . . 27
7.2.2 Code complet . . . 27
7.3 Exemple 2 : Système de commande avec réponse . . . 29
7.3.1 Scénario . . . 29
7.3.2 Code complet . . . 29
7.4 Exemple 3 : Test en boucle locale (sans matériel CAN) . . . 31
7.4.1 Scénario . . . 31
7.5 Exemple 4 : Debug / Monitoring CAN . . . 32
7.5.1 Scénario . . . 32
7.6 Checklist d'implémentation . . . 33

8 Guide d'Utilisation de TAGCAN **35**
8.1 Aperçu . . . 35
8.2 Structure du guide . . . 35
8.3 Points clés . . . 35
8.3.1 Qu'est-ce qu'un tag ? . . . 35
8.3.2 Architecture générale . . . 35
8.4 Démarrage rapide . . . 35
8.5 Documentation de référence . . . 36
8.6 Fichiers sources associés . . . 36

9 TAGCAN **37**

10 Data Structure Index **39**

10.1 Data Structures . . . 39

11 File Index **41**

11.1 File List . . . 41

12 Data Structure Documentation **43**

12.1 TagDef Struct Reference . . . 43

12.1.1 Detailed Description . . . 43

12.1.2 Field Documentation . . . 43

12.1.2.1 bit_pos_a . . . 43

12.1.2.2 bit_pos_b . . . 43

12.1.2.3 byte_idx_a . . . 43

12.1.2.4 byte_idx_b . . . 43

12.1.2.5 n_bits . . . 43

12.1.2.6 name . . . 44

12.2 tsfp Struct Reference . . . 44

12.2.1 Detailed Description . . . 44

12.2.2 Field Documentation . . . 44

12.2.2.1 bi . . . 44

13 File Documentation **45**

13.1 API_REFERENCE.md File Reference . . . 45

13.2 README.md File Reference . . . 45

13.3 README.md File Reference . . . 45

13.4 README.md File Reference . . . 45

13.5 com_utils.c File Reference . . . 45

13.5.1 Function Documentation . . . 46

13.5.1.1 CAN_change_val() . . . 46

13.5.1.2 CAN_init_TAGCAN() . . . 46

13.5.1.3 CAN_send_msg() . . . 46

13.5.1.4 CAN_setup_DF() . . . 46

13.5.1.5 CAN_setup_Filter() . . . 46

13.5.1.6 CAN_setup_TAG() . . . 46

13.5.1.7 HAL_CAN_RxFifo0MsgPendingCallback() . . . 46

13.5.1.8 init_com() . . . 46

13.5.1.9 main_loop() . . . 47

13.5.1.10 update_all_val_receve() . . . 47

13.5.1.11 update_all_val_send() . . . 47

13.5.1.12 update_TagData() . . . 47

13.5.1.13 update_val() . . . 47

13.5.2 Variable Documentation . . . 47

13.5.2.1 datacheck . . . 47

13.5.2.2 out_Rx . . . 47

13.5.2.3 RxData . . . 47
13.5.2.4 RxData_moteur . . . 47
13.5.2.5 RxHeader . . . 47
13.5.2.6 TxData_moteur . . . 47
13.5.2.7 TxHeader . . . 47
13.5.2.8 TxMailbox . . . 47
13.5.2.9 val_AB . . . 48
13.5.2.10 val_CI . . . 48
13.5.2.11 val_EDC . . . 48
13.5.2.12 val_HB . . . 48
13.5.2.13 val_N2ODVS . . . 48
13.5.2.14 val_N2OIV . . . 48
13.5.2.15 val_N2OMV . . . 48
13.5.2.16 val_N2OPS . . . 48
13.5.2.17 val_SEQ . . . 48
13.5.2.18 val_TIME . . . 48
13.5.2.19 val_TLC . . . 48
13.6 com_utils.h File Reference . . . 48
13.6.1 Function Documentation . . . 49
13.6.1.1 init_com() . . . 49
13.6.2 Variable Documentation . . . 49
13.6.2.1 hcan . . . 49
13.6.2.2 RxData . . . 49
13.6.2.3 RxHeader . . . 49
13.6.2.4 TxData . . . 49
13.6.2.5 TxHeader . . . 49
13.6.2.6 TxMailbox . . . 49
13.6.2.7 val_AB . . . 50
13.6.2.8 val_CI . . . 50
13.6.2.9 val_EDC . . . 50
13.6.2.10 val_HB . . . 50
13.6.2.11 val_N2ODVS . . . 50
13.6.2.12 val_N2OIV . . . 50
13.6.2.13 val_N2OMV . . . 50
13.6.2.14 val_N2OPS . . . 50
13.6.2.15 val_SEQ . . . 50
13.6.2.16 val_TIME . . . 50
13.6.2.17 val_TLC . . . 50
13.7 01_INITIALISATION.md File Reference . . . 50
13.8 02_GESTION_TAGS.md File Reference . . . 50
13.9 03_DONNEES_CAN.md File Reference . . . 50
13.10 04_FILTRAGE_CAN.md File Reference . . . 50

13.11 05_EXEMPLES_COMPLETS.md File Reference . . . 50
13.12 config_value.h File Reference . . . 50
13.12.1 Macro Definition Documentation . . . 51
13.12.1.1 MAX_N_GT . . . 51
13.12.1.2 MAX_TAG_NAME_LEN . . . 51
13.12.1.3 MAX_TAGS . . . 51
13.12.1.4 N_BYTES . . . 51
13.12.2 Enumeration Type Documentation . . . 52
13.12.2.1 CAN_MODE . . . 52
13.12.2.2 CAN_TG_STATUE . . . 52
13.12.3 Variable Documentation . . . 52
13.12.3.1 CAN_TG_ERROR_INTERNAL_CORUPTION . . . 52
13.12.3.2 CAN_TG_ERROR_NAME_ALREADY_USE . . . 52
13.12.3.3 CAN_TG_ERROR_NOT_ENOUNG_SPACE . . . 52
13.12.3.4 CAN_TG_ERROR_NULL_POINTOR . . . 53
13.12.3.5 CAN_TG_ERROR_SIZE_VALUE_INVALIDE . . . 53
13.12.3.6 CAN_TG_ERROR_SUBDIVISION_FAIL . . . 53
13.12.3.7 CAN_TG_ERROR_TAG_NAME_INVALIDE . . . 53
13.12.3.8 CAN_TG_ERROR_TAG_NAME_TO_BIGS . . . 53
13.12.3.9 CAN_TG_ERROR_TAG_NOT_FOUND . . . 53
13.12.3.10 CAN_TG_ERROR_TOO_MAY_TAGS . . . 53
13.12.3.11 CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER . . . 53
13.12.3.12 SUCCES . . . 53
13.12.3.13 SUCCES_TO_SET . . . 53
13.13 gestion_tag.h File Reference . . . 53
13.13.1 Function Documentation . . . 54
13.13.1.1 init_tag_manager() . . . 54
13.13.1.2 set_tag() . . . 54
13.14 data_container.h File Reference . . . 54
13.14.1 Function Documentation . . . 55
13.14.1.1 can_simulate_send_receive() . . . 55
13.14.1.2 get_value() . . . 55
13.14.1.3 init_data_container() . . . 56
13.14.1.4 prep_mask() . . . 56
13.14.1.5 set_val_TxData() . . . 56
13.14.1.6 set_value() . . . 56
13.14.2 Variable Documentation . . . 57
13.14.2.1 TxData . . . 57
13.15 print_val.h File Reference . . . 57
13.15.1 Function Documentation . . . 57
13.15.1.1 print_all_data() . . . 57
13.15.1.2 print_all_data_bin() . . . 58

13.15.1.3 print_all_tag_set() . . . 58
13.15.1.4 print_all_with_title() . . . 58
13.15.1.5 print_bin_8() . . . 58
13.15.1.6 print_repartition_bit() . . . 58
13.15.2 Variable Documentation . . . 59
13.15.2.1 gbc2 . . . 59
13.16 tag_manager.h File Reference . . . 59
13.16.1 Function Documentation . . . 59
13.16.1.1 get_tag_def() . . . 59
13.16.2 Variable Documentation . . . 60
13.16.2.1 g_bytes_used . . . 60
13.16.2.2 g_tag_count . . . 60
13.16.2.3 g_tags . . . 60
13.17 CAN_utils.c File Reference . . . 60
13.18 config_value.c File Reference . . . 60
13.18.1 Variable Documentation . . . 60
13.18.1.1 CAN_TG_ERROR_INTERNAL_CORUPTION . . . 60
13.18.1.2 CAN_TG_ERROR_NAME_ALREADY_USE . . . 60
13.18.1.3 CAN_TG_ERROR_NOT_ENOUNG_SPACE . . . 61
13.18.1.4 CAN_TG_ERROR_NULL_POINTOR . . . 61
13.18.1.5 CAN_TG_ERROR_SIZE_VALUE_INVALIDE . . . 61
13.18.1.6 CAN_TG_ERROR_SUBDIVISION_FAIL . . . 61
13.18.1.7 CAN_TG_ERROR_TAG_NAME_INVALIDE . . . 61
13.18.1.8 CAN_TG_ERROR_TAG_NAME_TO_BIGS . . . 61
13.18.1.9 CAN_TG_ERROR_TAG_NOT_FOUND . . . 61
13.18.1.10 CAN_TG_ERROR_TOO_MAY_TAGS . . . 61
13.18.1.11 CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER . . . 61
13.18.1.12 SUCCES . . . 61
13.18.1.13 SUCCES_TO_SET . . . 61
13.19 main.c File Reference . . . 62
13.19.1 Function Documentation . . . 62
13.19.1.1 main() . . . 62
13.19.1.2 run_all_tests() . . . 62
13.19.1.3 somme_other_test() . . . 62
13.19.1.4 TEST() . . . 62
13.19.1.5 TEST_EQ_INT() . . . 63
13.19.1.6 test_print() . . . 63
13.19.2 Variable Documentation . . . 63
13.19.2.1 passed_tests . . . 63
13.19.2.2 total_tests . . . 63
13.20 data_container.c File Reference . . . 63
13.20.1 Function Documentation . . . 64

13.20.1.1 can_simulate_send_receive() . . . 64
13.20.1.2 convert_8_to_16bit() . . . 64
13.20.1.3 get_value() . . . 64
13.20.1.4 init_data_container() . . . 65
13.20.1.5 prep_mask() . . . 65
13.20.1.6 set_val_TxData() . . . 65
13.20.1.7 set_value() . . . 65
13.20.2 Variable Documentation . . . 67
13.20.2.1 TxData . . . 67
13.21 print_val.c File Reference . . . 67
13.21.1 Function Documentation . . . 67
13.21.1.1 modif_vt() . . . 67
13.21.1.2 print_all_data() . . . 68
13.21.1.3 print_all_data_bin() . . . 68
13.21.1.4 print_all_tag_set() . . . 68
13.21.1.5 print_all_with_title() . . . 68
13.21.1.6 print_bin_8() . . . 68
13.21.1.7 print_repartition_bit() . . . 69
13.21.2 Variable Documentation . . . 69
13.21.2.1 gbc2 . . . 69
13.22 tag_manager.c File Reference . . . 69
13.22.1 Function Documentation . . . 70
13.22.1.1 find_byte2_with_enoung_space() . . . 70
13.22.1.2 find_byte_with_enoung_space() . . . 70
13.22.1.3 general_set_tag() . . . 70
13.22.1.4 get_first_tags_vide() . . . 71
13.22.1.5 get_tag_def() . . . 71
13.22.1.6 init_tag_manager() . . . 71
13.22.1.7 set_name_tag() . . . 71
13.22.1.8 set_new_tag_9_to_16() . . . 72
13.22.1.9 set_tag() . . . 72
13.22.2 Variable Documentation . . . 72
13.22.2.1 g_bytes_used . . . 72
13.22.2.2 g_tag_count . . . 72
13.22.2.3 g_tags . . . 72
13.23 TAGCAN.h File Reference . . . 72
13.24 test_print_repartition_bit.c File Reference . . . 73
13.24.1 Function Documentation . . . 73
13.24.1.1 main__() . . . 73
13.25 test_safty_func.c File Reference . . . 73
13.26 test_safty_func.h File Reference . . . 73
13.27 test_send_receve.c File Reference . . . 73

13.28 test_send_receve.h File Reference . . . 73
13.29 test_set_tag.c File Reference . . . 73
13.30 test_set_tag.h File Reference . . . 73
13.31 test_utils.c File Reference . . . 73
13.31.1 Function Documentation . . . 74
13.31.1.1 TEST_() . . . 74
13.31.1.2 TEST_EQ_INT_() . . . 74
13.31.2 Variable Documentation . . . 74
13.31.2.1 passed_tests . . . 74
13.31.2.2 total_tests . . . 74
13.32 test_utils.h File Reference . . . 74
13.32.1 Function Documentation . . . 74
13.32.1.1 TEST() . . . 74
13.32.1.2 TEST_EQ_INT() . . . 75
13.32.2 Variable Documentation . . . 75
13.32.2.1 passed_tests . . . 75
13.32.2.2 total_tests . . . 75
13.33 version.h File Reference . . . 75
13.33.1 Macro Definition Documentation . . . 75
13.33.1.1 RC_FILEVERSION . . . 75
13.33.1.2 RC_FILEVERSION_STRING . . . 75

Index **77**

Chapter 1

Documentation TAGCAN

Ce dossier contient une documentation manuelle (separee du code source) basee sur l'etat actuel du projet.

1.1 Contenu

- `documentation/API_REFERENCE.md`: reference des modules, structures, fonctions et codes de retour.
- `documentation/COMMENTAIRES_AUDIT.md`: verification des commentaires existants (qualite + ecarts).
- `documentation/COHERENCE.md`: verification de coherence entre code, commentaires et tests.

1.2 Perimetre

- Sources analysees: `include/`, `scripts/`, `tests/`.
- Date de verification: 2026-03-09.
- Objectif: documenter le comportement reel observe, puis signaler les incoherences.

Chapter 2

API Reference (Etat actuel)

2.1 Vue d'ensemble

TAGCAN gere un payload CAN de 8 octets (`TxData`) et permet:

- de declarer des tags (`set_tag`) avec une taille en bits,
- d'ecrire une valeur dans `TxData` via un tag (`set_value`),
- de relire une valeur via un tag (`get_value`).

2.2 Modules

- `include/config_value.h` + `scripts/config_value.c`: constantes et codes de retour.
- `include/gestion_tag.h` + `scripts/sub_scripts/tag_manager.c`: creation/gestion des tags.
- `include/sub_include/data_container.h` + `scripts/sub_scripts/data_container.c`↩ : stockage et acces aux valeurs.
- `include/sub_include/tag_manager.h`: structure `TagDef` et variables globales du gestionnaire.
- `include/sub_include/print_val.h` + `scripts/sub_scripts/print_val.c`: affichage/debug.

2.3 Structures et etat global

2.3.1 <tt>TagDef</tt> (<tt>include/sub_include/tag_manager.h</tt>)

- `name[MAX_TAG_NAME_LEN]`: nom logique du tag.
- `n_bits`: taille du tag en bits.
- `byte_idx_a`, `byte_idx_b`: index des octets utilises dans `TxData`.
- `bit_pos_a`, `bit_pos_b`: position des bits dans les octets.

2.3.2 Variables globales

- `TagDef g_tags[MAX_TAGS]`: table des tags.
- `uint8_t g_tag_count`: nombre de tags actifs.
- `uint8_t g_bytes_used[N_BYTES]`: occupation des octets.
- `uint8_t TxData[N_BYTES]`: buffer CAN (8 octets).

2.4 Fonctions principales

2.4.1 Gestion des tags

- `void init_tag_manager(void)`
  - Reinitialise `g_tags`, `g_tag_count` et `g_bytes_used`.
- `int set_tag(const char* tag_name, uint8_t n_bit)`
  - Declare un tag et lui alloue une zone dans `TxData`.
  - Implementation observee: accepte les tailles `1..16` (pas seulement `1, 3, 4, 8, 16`).
  - Retourne un code `CAN_TG_STATUE`.
- `const TagDef* get_tag_def(const char* tag_name)`
  - Recherche un tag par nom.
  - Retourne `NULL` si absent.

2.4.2 Valeurs (payload)

- `void init_data_container(void)`
  - Met `TxData` a zero.
- `int set_value(const char* tag_name, uint32_t value)`
  - Ecrit la valeur dans `TxData` selon la definition du tag.
  - Debordement detecte via `value >= (1ULL << tag->n_bits)`.
- `int get_value(const char* tag_name, uint32_t* out_value)`
  - Lit la valeur d'un tag depuis `TxData`.
- `uint8_t prep_mask(uint8_t p_bit, uint8_t n_bit)`
  - Fonction utilitaire de masque binaire.
- `void can_simulate_send_receive(uint8_t* tx_buffer, uint8_t* rx_buffer)`
  - Copie `tx_buffer` vers `rx_buffer` (simulation simple).
- `void set_val_TxData(uint8_t *TxDataArray)`
  - Copie un tableau externe vers `TxData`.

2.5 Debug / affichage

- `print_bin_8`, `print_all_data_bin`, `print_all_with_title`.
- `print_all_tag_set`, `print_all_data`, `print_repartition_bit`.

2.6 Codes de retour

2.6.1 Enum <tt>CAN_TG_STATUE</tt> (<tt>include/config_value.h</tt>)

- `CAN_TG_SUCCESS = 1`
- `CAN_TG_ERROR_NOT_ENOUNG_SPACE_ = -2`
- `CAN_TG_ERROR_TAG_NOT_FOUND_ = -3`
- `CAN_TG_ERROR_TAG_NAME_TO_BIGS_ = -4`
- `CAN_TG_ERROR_TOO_MAY_TAGS_ = -5`
- `CAN_TG_ERROR_SIZE_VALUE_INVALIDE_ = -6`
- `CAN_TG_ERROR_NAME_ALREADY_USE_ = -7`
- `CAN_TG_ERROR_SUBDIVISION_FAIL_ = -8`
- `CAN_TG_ERROR_TAG_NAME_INVALIDE_ = -9`
- `CAN_TG_ERROR_VALUE_TO_BIG = -10`
- `CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER_ = -21`
- `CAN_TG_ERROR_INTERNAL_CORUPTION_ = -30`

2.6.2 Constantes globales (<tt>scripts/config_value.c</tt>)

- Les constantes `CAN_TG_ERROR_*` existent aussi en variables globales.
- Attention: `CAN_TG_ERROR_NULL_POINTOR` vaut `-20` (constante), alors que `CAN_TG_ERROR_NULL↩_POINTOR_` vaut `-2` (enum).

2.7 Notes importantes de comportement

- Validation longueur nom de tag: implementation actuelle utilise `sizeof(tag_name)` (taille pointeur), pas `strlen(tag_name)`.
- Gestion debordement `set_value`: retourne `CAN_TG_ERROR_VALUE_TO_BIG` (`-10`).
- Les tests de `scripts/main.c` attendent parfois `-4` pour des debordements de valeur, ce qui diverge du code courant.

Chapter 3

01 - Initialisation de TAGCAN

3.1 Objectif

Configurer le système TAGCAN et le bus CAN pour pouvoir communiquer avec d'autres nœuds CAN.

3.2 Vue d'ensemble du processus

L'initialisation suit ces étapes :

```
1. init_com() (point d'appel unique)
   - CAN_init_TAGCAN()
   -  - init_data_container()     -> Efface le buffer TxData
   -  - init_tag_manager()        -> Initialise la gestion des tags
   -
   - CAN_setup_TAG()              -> Crée les tags applicatifs
   -  (set_tag pour chaque variable)
   -
   - CAN_setup_DF()               -> Configure le header CAN
   -  (ID, type de frame, DLC)
   -
   - HAL_CAN_ActivateNotification -> Active les interruptions RX
```

3.3 Détail de chaque étape

3.3.1 Étape 1 : init_data_container()

Réinitialise le buffer de données CAN à zéro.

```
void init_data_container(void);
```

Paramètre	Type	Description
-	-	(aucun)

Effet :

- Vide complètement le buffer `TxData[8]`
- Doit être appelé (au moins une fois) lors du démarrage

3.4 - Atention: un appel en cours d'exécution réinitialiseras les valeurs à 0.

3.4.1 Étape 2 : init_tag_manager()

Initialise le gestionnaire de tags et efface tous les tags existants.

```
void init_tag_manager(void);
```

Note : Après cet appel, tous les tags doivent être redéfinis avec `set_tag()`.

3.4.2 Étape 3 : CAN_setup_TAG()

Crée les tags applicatifs en fonction de vos besoins.

```
void CAN_setup_TAG(void) {
    set_tag("SEQ", 3);          // Compteur de séquences (3 bits)
    set_tag("HB", 1);           // Watchdog HeartBeat (1 bit)
    set_tag("CI", 2);           // Continuité igniter (2 bits)
    set_tag("AB", 7);           // Alim. Batterie en % (7 bits)
    set_tag("EDC", 4);          // États des commandes (4 bits)
    set_tag("N2OPS", 16);       // Capteur pression N2O (16 bits)
    set_tag("TLC", 12);         // Thrust Load Cell (12 bits)
    set_tag("N2ODVS", 1);       // État soupape Dump N2O (1 bit)
    set_tag("N2OIV", 1);        // Valve igniter N2O (1 bit)
    set_tag("N2OMV", 1);        // Soupape principale N2O (1 bit)
    set_tag("TIME", 16);        // Timestamp en ms (16 bits)
}
```

Paramètres de set_tag() :

```
int set_tag(const char* tag_name, uint8_t n_bit);
```

Paramètre	Type	Description
`tag_name`	`const char*`	Nom unique du tag (ex. "SEQ")
`n_bit`	`uint8_t`	Nombre de bits (1 à 16)

Valeur de retour :

- Code de succès ou code d'erreur
- À consulter pour débugger (voir `documentation/API_REFERENCE.md`)

3.4.3 Étape 4 : CAN_setup_DF()

Configure le header de transmission CAN (ID, type de frame, etc.).

```
void CAN_setup_DF(int32_t std_id) {
    std_id &= 0x7FFU;                // Masque pour ID standard 11 bits
    TxHeader.IDE = CAN_ID_STD;       // Utiliser ID standard (non étendu)
    TxHeader.StdId = std_id;         // Assigner l'ID CAN
    TxHeader.RTR = CAN_RTR_DATA;     // Mode données (non RTR)
    TxHeader.DLC = 8;                // Longueur = 8 octets
}
```

Points importants :

- `IDE = CAN_ID_STD` -> utilise les IDs standards (11 bits)
- `RTR = CAN_RTR_DATA` -> frame de données (pas une requête)
- `DLC = 8` -> longueur fixe du payload (8 octets)
- `StdId` -> l'identifiant unique du message CAN

3.4.4 Étape 5 : Activer les interruptions

```
if(HAL_CAN_ActivateNotification(&hcan, CAN_IT_RX_FIFO0_MSG_PENDING) != HAL_OK) {
    Error_Handler();
}
```

Explications :

- Permet de recevoir les messages CAN par interruption (plutôt que polling)
- `CAN_IT_RX_FIFO0_MSG_PENDING` -> déclenche une IRQ à la réception d'un message dans FIFO0
- Nécessite l'implémentation du callback : `void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef`

3.5 Code complet d'initialisation

```
#include "com_utils.h"
#include "TAGCAN.h"

void init_com(void) {
    // Étape 1 : Initialiser les conteneurs de données
    CAN_init_TAGCAN();

    // Étape 2 : Créer les tags applicatifs
    CAN_setup_TAG();

    // Étape 3 : Configurer le header CAN
    CAN_setup_DF(0x123);  // ID CAN = 0x123

    // Étape 4 : Activer la réception par interruption
    if(HAL_CAN_ActivateNotification(&hcan, CAN_IT_RX_FIFO0_MSG_PENDING) != HAL_OK) {
        Error_Handler();
    }

    //  Système TAGCAN prêt !
}

// À appeler une seule fois dans main() au démarrage
int main(void) {
    HAL_Init();
    MX_CAN_Init();      // Initialisation du module CAN matériel

    init_com();         // Initialisation TAGCAN

    // Boucle principale...
    while(1) {
        // Votre code
    }
}
```

3.6 Points d'attention

Point	Détail
Ordre d'appel	Toujours : `init_data_container()` -> `init_tag_manager()` -> `set_tag()`
Appel unique	`init_com()` doit être appelée **une seule fois** au démarrage
Réinitialisation	Si vous appelez `init_tag_manager()` à nouveau, tous les tags doivent être recréés
Configuration matérielle	Le module CAN STM32 (HAL) doit être initialisé **avant** TAGCAN
ID CAN unique	Chaque nœud du réseau doit avoir un ID CAN unique

3.7 Vérification

Après initialisation :

- `TxData[8]` est rempli de zéros
- Les tags sont créés et prêts à l'emploi
- Le header `TxHeader` est configuré avec votre ID CAN
- Les interruptions CAN sont actives
- Prêt pour `set_value()` et `get_value()`

Voir aussi :

- 02_GESTION_TAGS.md - Utiliser les tags
- 03_DONNEES_CAN.md - Gérer les données CAN

Chapter 4

02 - Gestion des Tags

4.1 Qu'est-ce qu'un tag ?

Un **tag** est une variable nommée qui occupe une plage de bits dans le buffer CAN (TxData). C'est l'interface principale pour lire et écrire des données.
Analogie : C'est comme des variables struct, mais mappées directement dans un buffer binaire.

4.2 Créer un tag : set_tag()

4.2.1 Prototype

```
int set_tag(const char* tag_name, uint8_t n_bit);
```

4.2.2 Paramètres

Paramètre	Type	Description
`tag_name`	`const char*`	Nom unique du tag (ex. "SEQ", "HB")
`n_bit`	`uint8_t`	Nombre de bits occupés (1 à 16)

4.2.3 Valeur de retour

Consulter `documentation/API_REFERENCE.md` pour les codes d'erreur.

4.2.4 Exemples

```
// Initialisation des tags (dans CAN_setup_TAG())
void CAN_setup_TAG(void) {
    set_tag("SEQ", 3);        // 3 bits pour numéro séquence
    set_tag("HB", 1);         // 1 bit pour watchdog
    set_tag("CI", 2);         // 2 bits pour continuité
    set_tag("AB", 7);         // 7 bits pour batterie (0-100%)
    set_tag("N2OPS", 16);     // 16 bits pour pression (0-65535 Pa)
    set_tag("TIME", 16);      // 16 bits pour timestamp (0-65535 ms)
}
```

4.3 Écrire une valeur : set_value()

4.3.1 Prototype

```
int set_value(const char* tag_name, uint32_t value);
```

4.3.2 Paramètres

Paramètre	Type	Description
`tag_name`	`const char*`	Nom du tag à modifier

Paramètre	Type	Description
`value`	`uint32_t`	Nouvelle valeur à écrire

4.3.3 Valeur de retour

- **CAN_TG_SUCCESS** (0) - Succès
- **CAN_TG_ERROR_VALUE_TO_BIG** - La valeur dépasse les `n_bit` bits du tag
- Autres codes d'erreur - Tag non trouvé, etc.

4.3.4 Comportement important

La valeur est tronquée si elle dépasse la capacité du tag.
Par exemple :

```
set_tag("SEQ", 3);       // 3 bits -> plage 0 à 7
set_value("SEQ", 10);    //  Erreur : 10 > 2^3 - 1 = 7
set_value("SEQ", 5);     //  Ok : écrit 0b101
```

4.3.5 Exemples d'utilisation

```
// Dans la boucle principale ou handler
static int seq_counter = 0;

void update_telemetry(void) {
    // Incrémenter et envoyer le numéro de séquence
    seq_counter = (seq_counter + 1) % 8;  // Cycle avec 3 bits (0-7)
    if(set_value("SEQ", seq_counter) != CAN_TG_SUCCESS) {
        // Erreur...
    }

    // Envoyer le statut batterie
    uint8_t battery_percent = read_battery();  // 0-100
    if(set_value("AB", battery_percent) != CAN_TG_SUCCESS) {
        // Erreur...
    }

    // Envoyer une pression
    uint16_t pressure = read_sensor();  // 0-65535
    if(set_value("N2OPS", pressure) != CAN_TG_SUCCESS) {
        // Erreur...
    }

    // Envoyer le timestamp
    uint16_t time_ms = get_timestamp();
    set_value("TIME", time_ms);
}
```

4.4 Lire une valeur : get_value()

4.4.1 Prototype

```
int get_value(const char* tag_name, uint32_t* out_value);
```

4.4.2 Paramètres

Paramètre	Type	Description
`tag_name`	`const char*`	Nom du tag à lire
`out_value`	`uint32_t*`	Pointeur vers variable de sortie

4.4.3 Valeur de retour

- **CAN_TG_SUCCESS** (0) - Succès, `*out_value` est valide
- Négatif - Code d'erreur

4.4.4 Exemples

```
uint32_t val_seq;
if(get_value("SEQ", &val_seq) == CAN_TG_SUCCESS) {
    printf("Séquence: %u\n", val_seq);
}

uint32_t val_battery;
if(get_value("AB", &val_battery) == CAN_TG_SUCCESS) {
    printf("Batterie: %u %%\n", val_battery);
}

uint32_t val_pressure;
if(get_value("N2OPS", &val_pressure) == CAN_TG_SUCCESS) {
    printf("Pression N2O: %u Pa\n", val_pressure);
}
```

4.5 Workflow complet : set / get / send

4.5.1 Scénario d'exemple

```
#include "TAGCAN.h"
#include "stm32f1xx_hal.h"

CAN_HandleTypeDef hcan;
uint8_t TxData[8];
CAN_TxHeaderTypeDef TxHeader;
int32_t TxMailbox;

// Étape 1: Initialisation (une fois au démarrage)
void init_system(void) {
    init_com();  // Appelle init_data_container(), init_tag_manager(), set_tag(), etc.
}

// Étape 2: Boucle de mise à jour périodique
void periodic_10ms_task(void) {
    static uint8_t seq = 0;

    // Préparer les données
    seq = (seq + 1) % 8;
    set_value("SEQ", seq);
    set_value("HB", 1);  // Heartbeat actif
    set_value("AB", read_adc_battery());
    set_value("EDC", get_command_state());
    set_value("TIME", HAL_GetTick());

    // Envoyer le frame CAN
    if(HAL_CAN_AddTxMessage(&hcan, &TxHeader, TxData, &TxMailbox) != HAL_OK) {
        // Erreur d'envoi...
    }
}

// Étape 3: Callback de réception (déclenchée par interruption)
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
    CAN_RxHeaderTypeDef RxHeader;
    uint8_t RxData[8];

    // Recevoir le mensaje
    if(HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxData) == HAL_OK) {
        // Copier dans le buffer de TAGCAN
        can_simulate_send_receive(RxData, NULL);  // ou autre traitement

        // Lire les valeurs reçues
        uint32_t remote_seq, remote_battery;
        get_value("SEQ", &remote_seq);
        get_value("AB", &remote_battery);

        printf("Nœud distant: SEQ=%u, Batterie=%u%%\n", remote_seq, remote_battery);
    }
}

int main(void) {
    HAL_Init();
    MX_CAN_Init();
    init_system();

    while(1) {
        periodic_10ms_task();
        // ...
    }
}
```

4.6 Bonnes pratiques

À faire	À éviter
Vérifier la valeur de retour de `set_value()`	Ignorer les codes d'erreur
Vérifier la valeur de retour de `get_value()`	Utiliser `out_value` sans vérification
Nommer les tags de façon explicite	Utiliser des noms génériques ("VAL1", "VAL2")
Dimensionner les tags selon les besoins	Allouer trop de bits (gaspillage)
Prévoir le cycle des valeurs (ex: seq 0-7 avec 3 bits)	Dépasser la limite d'un tag sans gérer
Documenter votre schéma de tags	Laisser du code sans commentaires

4.7 Schéma de tags recommandé

Pour un système similaire à l'exemple, structurez les tags comme ceci en commentaire :

```
/*
 * SCHÉMA DE TAGS CAN (ID 0x123)
 * ----------------------------------------
 * Byte 0: [HB(1) | SEQ(3) | CI(2) | XX(2)]
 * Byte 1: [AB(7) | YY(1)]
 * Byte 2-3: [EDC(4) | PADDING(12)]
 * Byte 4-5: [N2OPS(16)]        -- Pression N2O (Pa)
 * Byte 6-7: [TLC(12) | N2ODVS(1) | N2OIV(1) | N2OMV(1) | XX(1)]
 *
 * Total: 15 bits utilisés, reste pour extension
 */
```

Voir aussi :

- 03_DONNEES_CAN.md - Utilisation avancée du buffer
- 05_EXEMPLES_COMPLETS.md - Exemples concrets

Chapter 5

03 - Manipulation du Buffer de Données CAN

5.1 Objectif

Comprendre comment gérer le buffer de données CAN (`TxData` et `RxData`) au-delà de la simple interface des tags.

5.2 Structure du buffer CAN

5.2.1 Taille et format

```
uint8_t TxData[8];    // Buffer de transmission (8 octets = CAN standard)
uint8_t RxData[8];    // Buffer de réception (8 octets)
```

Le buffer CAN est toujours de **8 octets** (64 bits) pour un frame standard CAN.

5.2.2 Hiérarchie

Niveau	Composant	Responsabilité
Haut	Application	Appelle `set_value()`, `get_value()`
Moyen	Data Container (TAGCAN)	Mappe les tags dans le buffer
Bas	Buffer `TxData[8]` / `RxData[8]`	Storiage binaire brut
Très bas	HAL CAN STM32	Envoi/réception matérielle

5.3 Fonctions de gestion du buffer

5.3.1 1 init_data_container()

Réinitialise le buffer à zéro.

```
void init_data_container(void);
```

Effet :

- Tous les octets de `TxData[8]` -> 0
- Appelé manuellement si vous voulez "effacer" les données

Exemple :

```
// Avant:  TxData = {0x12, 0x34, 0x56, ...}
init_data_container();
// Après:  TxData = {0x00, 0x00, 0x00, ...}
```

5.3.2 2 set_val_TxData()

Copie un tableau **externe** dans le buffer global `TxData`.

```
void set_val_TxData(uint8_t *TxDataArray);
```

Utile pour :

- Charger des données pré-formatées
- Importer des données d'une autre source
- Tester sans passer par les tags

Exemple :

```
uint8_t raw_data[8] = {0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF, 0x00, 0x11};
set_val_TxData(raw_data);  // Copie raw_data -> TxData

// Après cet appel:
// TxData[0] = 0xAA
// TxData[1] = 0xBB
// ... et cetera

//  Les tags ne voient PAS cette modification directement
// Vous travaillez maintenant "en dehors" du système de tags
```

5.3.3 3 can_simulate_send_receive()

Simule un envoi/réception CAN par copie buffer-à-buffer.

```
void can_simulate_send_receive(uint8_t* tx_buffer, uint8_t* rx_buffer);
```

Cas d'usage :

- Tests de la logique sans matériel CAN
- Vérifier que les données écrites sont bien formatées
- Boucle locale pour déboguer

Exemple :

```
uint8_t tx[8] = {0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00};
uint8_t rx[8];

// Écrire des données dans tx
set_value("SEQ", 3);
can_simulate_send_receive(TxData, rx);  // Simule envoi/réception

// rx contient maintenant une copie de TxData
memcpy(rx, TxData, 8);  // Équivalent à:
                         // can_simulate_send_receive(TxData, rx);
```

5.4 Workflow complet : Écriture -> Envoi -> Réception

5.4.1 Émission (Tx)

```
Votre application
        ↓
    set_value("SEQ", 5)
    set_value("AB", 75)
    ...
        ↓
   TxData[8] (rempli par TAGCAN)
        ↓
   Transmission CAN (HAL)
```

5.4.2 Code d'émission

```
#include "TAGCAN.h"

extern CAN_HandleTypeDef hcan;
extern uint8_t TxData[8];
extern CAN_TxHeaderTypeDef TxHeader;
extern int32_t TxMailbox;

void send_can_frame(void) {
    // Étape 1: Réinitialiser le buffer (optionnel)
    // init_data_container();  // Si vous voulez un buffer vierge

    // Étape 2: Écrire les données via les tags
    set_value("SEQ", 5);
    set_value("HB", 1);
    set_value("AB", 75);
    set_value("N2OPS", 2500);

    // Étape 3: Envoyer via CAN
    HAL_Status = HAL_CAN_AddTxMessage(&hcan, &TxHeader, TxData, &TxMailbox);
    if(HAL_Status != HAL_OK) {
```

```
        printf("Erreur d'envoi CAN\n");
    }
}
```

5.4.3 Réception (Rx)

```
   Bus CAN
      ↓
Réception CAN (HAL)
      ↓
   RxData[8]
      ↓
 Votre application récupère les données
 via get_value() ou directement dans RxData[8]
```

5.4.4 Code de réception

```
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
    CAN_RxHeaderTypeDef RxHeader;
    uint8_t RxData[8];

    // Étape 1: Recevoir le message via HAL
    if(HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxData) != HAL_OK) {
        return;  // Erreur
    }

    // Étape 2a: Utiliser les données directement (acccès brut)
    printf("Octet 0: 0x%02X\n", RxData[0]);
    printf("Octet 1: 0x%02X\n", RxData[1]);

    // OU

    // Étape 2b: Interpréter via les tags
    // (si vous avez le même schéma de tags côté récepteur)
    uint32_t seq, battery;
    if(RxData[0] & 0x38) {  // Bits 3-5 contiennent possibly SEQ
        // Extraire manuellement...
    }

    // Ou utiliser les fonctions TAGCAN si vous avez copié RxData dans TxData:
    memcpy(TxData, RxData, 8);  // Copier RxData -> TxData
    if(get_value("SEQ", &seq) == CAN_TG_SUCCESS) {
        printf("Numéro de séquence distant: %u\n", seq);
    }
}
```

5.5 Directement lire/écrire les octets

5.5.1 Lecture directe

```
// Accéder directement à un octet
uint8_t byte0 = TxData[0];  // Premier octet
uint8_t byte7 = TxData[7];  // Dernier octet

// Accéder à un bit spécifique
uint8_t bit3 = (TxData[0] » 3) & 1;  // Bit 3 de l'octet 0
```

5.5.2 Écriture directe

```
// Écrire un octet entier
TxData[0] = 0xA5;  // Écrit 0b10100101

// Modifier un bit spécifique
TxData[0] |= (1 « 3);   // Mettre à 1 le bit 3
TxData[0] &= ~(1 « 3);  // Mettre à 0 le bit 3

// Bitmask pour plusieurs bits
TxData[0] = (TxData[0] & 0xF0) | 0x05;  // Garder les 4 bits hauts, définir bas à 0x05
```

5.6 Pièges courants

Piège	Description	Solution
Mélanger accès direct et tags	Écrire un octet directement, puis utiliser `get_value()` peut être confus	Soyez cohérent: ou tags, ou accès brut

Piège	Description	Solution
Oublier le memcpy ()	Recevoir RxData, mais essayer de lire avec get_value() sans copier	Toujours copier RxData -> Tx↩ Data avant get_value()
Buffer réutilisé	Si set_value() change l'ordre de allocation des tags	Pas vraiment un piège, mais attention si tags réordonnés
Débordement de bits	Écrire un octet directement sans respecter le schéma de tags	Utilisez print_repartition_bit() (déclarée dans include/sub_include/print_val.h) pour visualiser exactement le layout des tags.

5.7 Tableau d'aide : Qui fait quoi ?

Tâche	Fonction	Détails
Vider le buffer	init_data_container()	Remplace TxData par des zéros
Écrire une valeur nommée	set_value(tag, val)	Via l'interface des tags
Lire une valeur nommée	get_value(tag, &val)	Via l'interface des tags
Charger des données brutes	set_val_TxData(arr)	Copie un tableau externe
Tester localement	can_simulate_send_receive()	Simule une boucle locale
Envoyer sur le bus	HAL_CAN_AddTxMessage()	Transport matériel (HAL STM32)
Recevoir du bus	HAL_CAN_GetRxMessage()	Transport matériel (HAL STM32)

5.8 Exemple : Envoi -> Réception -> Traitement

```
#include "TAGCAN.h"
#include "stm32f1xx_hal.h"

extern CAN_HandleTypeDef hcan;
extern uint8_t TxData[8], RxData[8];
extern CAN_TxHeaderTypeDef TxHeader;
extern int32_t TxMailbox;

// Tâche d'émission (toutes les 10ms)
void task_tx_10ms(void) {
    static uint8_t seq = 0;
    seq = (seq + 1) % 8;

    // Écrire les données
    set_value("SEQ", seq);
    set_value("HB", 1);
    set_value("AB", read_battery_percent());

    // Envoyer
    HAL_CAN_AddTxMessage(&hcan, &TxHeader, TxData, &TxMailbox);
}

// Callback de réception (IRQ)
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
    CAN_RxHeaderTypeDef RxHeader;
    uint8_t RxDataLocal[8];

    HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxDataLocal);

    // Voir directement les octets reçus
    printf("Reçu: ");
    for(int i = 0; i < 8; i++) {
        printf("%02X ", RxDataLocal[i]);
    }
    printf("\n");

    // Si le même schéma de tags : interpréter via TAGCAN
    memcpy(TxData, RxDataLocal, 8);
    uint32_t remote_seq, remote_battery;
    get_value("SEQ", &remote_seq);
    get_value("AB", &remote_battery);
    printf("Séquence: %u, Batterie: %u%%\n", remote_seq, remote_battery);
```

```
}

int main(void) {
    HAL_Init();
    MX_CAN_Init();
    init_com();

    // Timer pour task_tx_10ms()
    // ...

    while(1) {
        // Boucle principale
    }
}
```

Voir aussi :

- 02_GESTION_TAGS.md - Tags et variables nommées
- 04_FILTRAGE_CAN.md - Filtrage des messages reçus

Chapter 6

04 - Filtrage CAN

6.1 Objectif

Configurer les filtres CAN pour recevoir **uniquement** les messages pertinents et ignorer le reste du trafic.

6.2 Contexte

Le bus CAN est partagé. Un nœud peut recevoir des messages destinés à d'autres nœuds. Les **filtres CAN** permettent :

- Réduire la charge CPU (moins d'interruptions inutiles)
- Améliorer la latence (traiter uniquement les messages pertinents)
- Séparer logiquement le trafic

6.3 Fonction de filtrage

6.3.1 Prototype

```
CAN_FilterTypeDef* CAN_setup_Filter(int32_t minimum_id, int32_t maximum_id, int filterBank);
```

6.3.2 Paramètres

Paramètre	Type	Description
`minimum_id`	`int32_t`	ID CAN minimum à accepter
`maximum_id`	`int32_t`	ID CAN maximum à accepter
`filterBank`	`int`	Numéro de banc de filtre (0-27 disponibles)

6.3.3 Valeur de retour

Pointeur vers la structure `CAN_FilterTypeDef` configurée.

6.4 Comment ça marche ?

6.4.1 Le filtre masque (IDMASK)

```
CAN_FilterTypeDef canfilterconfig;
canfilterconfig.FilterMode = CAN_FILTERMODE_IDMASK;  // Mode masque
```

Mode masque : Les bits du filtre agissent comme un masque binaire.

```
ID reçu :      0011 0101
Filtre :       1111 1111  (masque)
Résultat :     Accepter si bits importants Match
```

6.5 Exemple de configuration

6.5.1 Cas 1 : Accepter une plage d'IDs (0x100 à 0x120)

```
CAN_FilterTypeDef* filter1 = CAN_setup_Filter(0x100, 0x120, 0);
// Accepte les IDs: 0x100, 0x101, 0x102, ..., 0x120
```

6.5.2 Cas 2 : Accepter un ID exact (0x123)

```
CAN_FilterTypeDef* filter2 = CAN_setup_Filter(0x123, 0x123, 1);
// Accepte UNIQUEMENT: 0x123
```

6.5.3 Cas 3 : Accepter tous les messages

```
CAN_FilterTypeDef* filter3 = CAN_setup_Filter(0x000, 0x7FF, 2);
// Accepte les IDs: 0x000 à 0x7FF (tous les IDs standards 11 bits)
```

6.6 Détails de la configuration interne

Voici ce que fait `CAN_setup_Filter()` en détail :

```
CAN_FilterTypeDef* CAN_setup_Filter(int32_t minimum_id, int32_t maximum_id, int filterBank) {
    CAN_FilterTypeDef canfilterconfig;

    // Activer le filtre
    canfilterconfig.FilterActivation = CAN_FILTER_ENABLE;

    // Sélectionner le numéro de banc (0-27)
    canfilterconfig.FilterBank = filterBank;

    // Affecter à FIFO0 (interrupt CAN_IT_RX_FIFO0_MSG_PENDING)
    canfilterconfig.FilterFIFOAssignment = CAN_FILTER_FIFO0;

    // Configurer les seuils ID haut et bas
    canfilterconfig.FilterIdHigh = maximum_id « 5;    // Maximum
    canfilterconfig.FilterMaskIdHigh = maximum_id « 5; // Masque

    canfilterconfig.FilterIdLow = minimum_id;           // Minimum
    canfilterconfig.FilterMaskIdLow = minimum_id;       // Masque

    // Mode de filtrage (masque binaire)
    canfilterconfig.FilterMode = CAN_FILTERMODE_IDMASK;

    // Résolution: 32 bits (combinaison High + Low)
    canfilterconfig.FilterScale = CAN_FILTERSCALE_32BIT;

    // Assigner les bancs de filtrage au CAN1 (non CAN2)
    canfilterconfig.SlaveStartFilterBank = 20;

    // Appliquer la configuration matérielle
    HAL_CAN_ConfigFilter(&hcan, &canfilterconfig);

    return &canfilterconfig;
}
```

6.7 Exemple d'intégration complet

6.7.1 Initialisation avec filtrage

```
#include "TAGCAN.h"
#include "stm32f1xx_hal.h"

extern CAN_HandleTypeDef hcan;

void init_com(void) {
    // 1. Initialiser TAGCAN
    CAN_init_TAGCAN();
    CAN_setup_TAG();
    CAN_setup_DF(0x123);  // Mon ID CAN

    // 2. Configurer les filtres
    // Filtre 1: Accepter les messages du capteur de pression (0x200)
    CAN_setup_Filter(0x200, 0x200, 0);

    // Filtre 2: Accepter les messages de commande (0x100 à 0x110)
    CAN_setup_Filter(0x100, 0x110, 1);

    // Filtre 3: Messages d'urgence (0x050)
```

```
    CAN_setup_Filter(0x050, 0x050, 2);

    // 3. Activer les interruptions
    if(HAL_CAN_ActivateNotification(&hcan, CAN_IT_RX_FIFO0_MSG_PENDING) != HAL_OK) {
        Error_Handler();
    }
}

void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
    CAN_RxHeaderTypeDef RxHeader;
    uint8_t RxData[8];

    HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxData);

    // Traiter selon l'ID reçu
    switch(RxHeader.StdId) {
        case 0x200:
            handle_pressure_sensor(RxData);
            break;
        case 0x100:
        case 0x101:
        case 0x102:
        // ... jusqu'à 0x110
            handle_command(RxData);
            break;
        case 0x050:
            handle_emergency(RxData);
            break;
        default:
            // Ne devrait jamais arriver ici grâce aux filtres
            break;
    }
}
```

6.8 Limitation des bancs de filtrage

Le STM32F1 propose :

Ressource	Limite
Bancs de filtrage disponibles	28 (0-27)
Bancs réservés à CAN1	14 (0-13)
Bancs réservés à CAN2	14 (14-27)

Chaque filtre peut gérer :

- **1 ID exact** (FilterMode = IDMASK)
- **Une plage d'IDs** (FilterMode = IDLIST)

6.9 Stratégies de filtrage recommandées

6.9.1 Stratégie 1 : Filtrage par catégorie (Recommandé)

```
// Tous les capteurs: 0x200-0x20F (1 filtre)
CAN_setup_Filter(0x200, 0x20F, 0);

// Toutes les commandes: 0x100-0x10F (1 filtre)
CAN_setup_Filter(0x100, 0x10F, 1);

// Tout le diagnostic: 0x300-0x3FF (1 filtre)
CAN_setup_Filter(0x300, 0x3FF, 2);
```

Avantages :

- Utilise peu de filtres
- Facile à maintenir
- Extensible

6.9.2 Stratégie 2 : Un filtre par source

```
// Capteur 1: 0x200
CAN_setup_Filter(0x200, 0x200, 0);

// Capteur 2: 0x201
CAN_setup_Filter(0x201, 0x201, 1);

// Capteur 3: 0x202
CAN_setup_Filter(0x202, 0x202, 2);

// ... etc
```

Avantages :

- Granularité fine
- Gestion précise

Inconvénients :

- Consomme rapidement les bancs de filtrage

6.10 Points d'attention

Point	Détail
Ordre d'application	Les filtres doivent être configurés AVANT d'activer les interruptions
Limite de bancs	14 filtres max pour CAN1. Bien planifier!
FIFO0 vs FIFO1	`CAN_setup_Filter()` utilise FIFO0. À adapter si vous utilisez FIFO1
Pas de filtre = tout accepter	Si aucun filtre n'est configuré, tous les messages passent
Mode IDLIST vs IDMASK	La fonction utilise IDMASK (plages). Pour listes, modifier le code
Décalage des bits (<<5)	Note: `maximum_id << 5` -> convertit l'ID de 11 bits au format STM32

6.11 Débuggage des filtres

```
// Vérifier quels messages arrivent
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
    CAN_RxHeaderTypeDef RxHeader;
    uint8_t RxData[8];

    HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxData);

    // Afficher le message reçu
    printf("ID: 0x%03X, DLC: %u\n", RxHeader.StdId, RxHeader.DLC);
    for(int i = 0; i < RxHeader.DLC; i++) {
        printf("  Data[%d] = 0x%02X\n", i, RxData[i]);
    }
}
```

Si vous ne voyez **aucun message** malgré l'activité CAN :

- Vérifier que les filtres acceptent l'ID envoyé
- Vérifier que les interruptions sont activées
- Vérifier que le module CAN est initialisé

Si vous voyez **trop de messages** :

- Ajouter des filtres plus restrictifs
- Vérifier que les filtres ne sont pas inversés (min > max)

6.12 Comparaison : Avec/Sans filtres

6.12.1 Sans filtres (accepte tout)

```
// Aucun CAN_setup_Filter() appelé
while(1) {
    //  Reçoit TOUS les messages du bus
    // -> CPU chargé, latence mauvaise
}
```

6.12.2 Avec filtres (recommandé)

```
// Dans init_com()
CAN_setup_Filter(0x200, 0x20F, 0);  // Capteurs
CAN_setup_Filter(0x100, 0x10F, 1);  // Commandes

while(1) {
    //  Reçoit UNIQUEMENT les messages 0x200-0x20F et 0x100-0x10F
    // -> CPU libre, latence optimale
}
```

Voir aussi :

- 03_DONNEES_CAN.md - Buffer de données
- 05_EXEMPLES_COMPLETS.md - Système complet

Chapter 7

05 - Exemples Complets

7.1 Exemples d'intégration pratiques

Vous trouverez ici des exemples réalistes exploitant l'ensemble du système TAGCAN.

7.2 Exemple 1 : Système de télémétrie simple

7.2.1 Scénario

Un véhicule envoie périodiquement :

- Son numéro de séquence (3 bits)
- Son état (heartbeat, 1 bit)
- Le % de batterie (7 bits)
- La pression du réservoir (16 bits)
- Un timestamp (16 bits)

7.2.2 Code complet

```
// ============================================================================
// telemetrie.h
// ============================================================================

#ifndef TELEMETRIE_H_
#define TELEMETRIE_H_

#include "TAGCAN.h"
#include <stdint.h>

typedef struct {
    uint8_t seq;
    uint8_t heartbeat;
    uint8_t battery_percent;
    uint16_t reservoir_pressure;
    uint16_t timestamp_ms;
} telemetry_data_t;

// Initialiser le système de télémétrie
void telemetry_init(void);

// Envoyer un paquet de télémétrie
void telemetry_send(const telemetry_data_t *data);

// Traiter un paquet reçu
void telemetry_handle_rx(const uint8_t rx_data[8]);

#endif /* TELEMETRIE_H_ */


// ============================================================================
// telemetrie.c
// ============================================================================

#include "telemetrie.h"
```

```c
#include "stm32f1xx_hal.h"

extern CAN_HandleTypeDef hcan;
extern uint8_t TxData[8], RxData[8];
extern CAN_TxHeaderTypeDef TxHeader;
extern int32_t TxMailbox;

#define TELEMETRY_CAN_ID 0x100

void telemetry_init(void) {
    // Initialiser TAGCAN
    init_data_container();
    init_tag_manager();

    // Créer les tags
    set_tag("SEQ", 3);          // Numéro de séquence (0-7)
    set_tag("HB", 1);           // Heartbeat (0 ou 1)
    set_tag("BATTERY", 7);      // % batterie (0-100)
    set_tag("PRESSURE", 16);    // Pression réservoir (0-65535 Pa)
    set_tag("TIME", 16);        // Timestamp (0-65535 ms)

    // Configurer le header CAN
    TxHeader.IDE = CAN_ID_STD;
    TxHeader.StdId = TELEMETRY_CAN_ID;
    TxHeader.RTR = CAN_RTR_DATA;
    TxHeader.DLC = 8;

    // Configurer le filtre pour recevoir les télémétries des autres (ex: 0x101-0x10F)
    CAN_setup_Filter(0x101, 0x10F, 0);

    // Activer les interruptions
    if(HAL_CAN_ActivateNotification(&hcan, CAN_IT_RX_FIFO0_MSG_PENDING) != HAL_OK) {
        Error_Handler();
    }

    printf("[Telemetry] Initialized on CAN ID 0x%03X\n", TELEMETRY_CAN_ID);
}

void telemetry_send(const telemetry_data_t *data) {
    // Réinitialiser le buffer
    init_data_container();

    // Écrire les données
    set_value("SEQ", data->seq);
    set_value("HB", data->heartbeat);
    set_value("BATTERY", data->battery_percent);
    set_value("PRESSURE", data->reservoir_pressure);
    set_value("TIME", data->timestamp_ms);

    // Envoyer sur le bus
    if(HAL_CAN_AddTxMessage(&hcan, &TxHeader, TxData, &TxMailbox) != HAL_OK) {
        printf("[Telemetry] ERROR: Failed to send CAN message\n");
    } else {
        printf("[Telemetry] TX: SEQ=%u, BAT=%u%%, PRESSURE=%u Pa\n",
               data->seq, data->battery_percent, data->reservoir_pressure);
    }
}

void telemetry_handle_rx(const uint8_t rx_data[8]) {
    // Copier les données reçues dans TxData pour utiliser TAGCAN
    memcpy(TxData, rx_data, 8);

    // Lire via les tags
    uint32_t seq, hb, battery, pressure, timestamp;
    get_value("SEQ", &seq);
    get_value("HB", &hb);
    get_value("BATTERY", &battery);
    get_value("PRESSURE", &pressure);
    get_value("TIME", &timestamp);

    printf("[Telemetry] RX: SEQ=%u, HB=%u, BAT=%u%%, PRESSURE=%u Pa, TIME=%u ms\n",
           (uint8_t)seq, (uint8_t)hb, (uint8_t)battery, (uint16_t)pressure, (uint16_t)timestamp);
}

// ============================================================================
// Callback d'interruption CAN
// ============================================================================

void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
    CAN_RxHeaderTypeDef RxHeader;
    uint8_t RxDataLocal[8];

    if(HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxDataLocal) == HAL_OK) {
        // Traiter selon l'ID CAN reçu
        if((RxHeader.StdId >= 0x101) && (RxHeader.StdId <= 0x10F)) {
            telemetry_handle_rx(RxDataLocal);
        }
```

```
    }
}

// ==========================================================================
// main.c - Boucle principale
// ==========================================================================

// Tâche d'émission (10 ms)
void task_telemetry_10ms(void) {
    static uint8_t seq = 0;
    static uint32_t last_time = 0;

    telemetry_data_t data = {
        .seq = (seq++) % 8,
        .heartbeat = 1,
        .battery_percent = read_adc_battery(),
        .reservoir_pressure = read_sensor_pressure(),
        .timestamp_ms = HAL_GetTick() & 0xFFFF
    };

    if((HAL_GetTick() - last_time) >= 10) {
        telemetry_send(&data);
        last_time = HAL_GetTick();
    }
}

int main(void) {
    HAL_Init();
    MX_CAN_Init();

    telemetry_init();

    // Timer pour appeler task_telemetry_10ms() tous les 10ms
    // ...

    while(1) {
        task_telemetry_10ms();
    }
}
```

7.3 Exemple 2 : Système de commande avec réponse

7.3.1 Scénario

- **Maître envoie commandes** sur ID 0x200 (allumer/éteindre servos)
- **Esclaves répondent** sur ID 0x201-0x203 (confirmation + état)

7.3.2 Code complet

```
// ==========================================================================
// commandes.h
// ==========================================================================

#ifndef COMMANDES_H_
#define COMMANDES_H_

#include "TAGCAN.h"
#include <stdint.h>

// IDs CAN
#define CMD_CAN_ID_TX       0x200
#define CMD_CAN_ID_RX_BASE  0x201

// Commandes possibles
typedef enum {
    CMD_SERVO_OFF = 0,
    CMD_SERVO_ON = 1,
    CMD_SERVO_PULSE = 2
} cmd_type_t;

typedef struct {
    uint8_t device_id;      // Quel servomoteur ? (1-3)
    cmd_type_t command;     // Quelle commande ?
    uint8_t parameter;      // Paramètre optionnel (ex: pulse width)
} command_t;

typedef struct {
    uint8_t device_id;
    uint8_t status;         // 0=erreur, 1=ok
    uint8_t feedback;       // Retour d'état
```

```
} response_t;

void commands_init(void);
void commands_send(const command_t *cmd);
void commands_handle_response(uint8_t device_id, const uint8_t rx_data[8]);

#endif


// ============================================================================
// commandes.c
// ============================================================================

#include "commandes.h"
#include "stm32f1xx_hal.h"

extern CAN_HandleTypeDef hcan;
extern uint8_t TxData[8];
extern CAN_TxHeaderTypeDef TxHeader;
extern int32_t TxMailbox;

void commands_init(void) {
    // Initialiser TAGCAN
    init_data_container();
    init_tag_manager();

    // Tags pour les commandes ENVOYÉES
    set_tag("DEV_ID", 2);       // ID du périphérique (0-3)
    set_tag("CMD_TYPE", 2);     // Type de commande (0-3)
    set_tag("PARAM", 8);        // Paramètre (0-255)

    // Configuration du header TX
    TxHeader.IDE = CAN_ID_STD;
    TxHeader.StdId = CMD_CAN_ID_TX;
    TxHeader.RTR = CAN_RTR_DATA;
    TxHeader.DLC = 8;

    // Filtre pour les réponses des esclaves (0x201-0x203)
    CAN_setup_Filter(0x201, 0x203, 0);

    // Interruptions
    if(HAL_CAN_ActivateNotification(&hcan, CAN_IT_RX_FIFO0_MSG_PENDING) != HAL_OK) {
        Error_Handler();
    }

    printf("[Commands] Initialized\n");
}

void commands_send(const command_t *cmd) {
    init_data_container();

    set_value("DEV_ID", cmd->device_id);
    set_value("CMD_TYPE", cmd->command);
    set_value("PARAM", cmd->parameter);

    if(HAL_CAN_AddTxMessage(&hcan, &TxHeader, TxData, &TxMailbox) != HAL_OK) {
        printf("[Commands] ERROR: Send failed\n");
    } else {
        printf("[Commands] TX: Device=%u, Command=%u, Param=%u\n",
               cmd->device_id, cmd->command, cmd->parameter);
    }
}

void commands_handle_response(uint8_t device_id, const uint8_t rx_data[8]) {
    memcpy(TxData, rx_data, 8);

    uint32_t status, feedback;
    get_value("DEV_ID", NULL);  // Ou extraire directement
    get_value("STATUS", &status);
    get_value("FEEDBACK", &feedback);

    printf("[Commands] RX Response from Device %u: Status=%u, Feedback=%u\n",
           device_id, (uint8_t)status, (uint8_t)feedback);
}

void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
    CAN_RxHeaderTypeDef RxHeader;
    uint8_t RxDataLocal[8];

    if(HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxDataLocal) == HAL_OK) {
        uint8_t device_id = RxHeader.StdId - CMD_CAN_ID_RX_BASE + 1;
        if((RxHeader.StdId >= 0x201) && (RxHeader.StdId <= 0x203)) {
            commands_handle_response(device_id, RxDataLocal);
        }
    }
```

```
}

// ==========================================================================
// main.c - Exemple d'utilisation
// ==========================================================================

int main(void) {
    HAL_Init();
    MX_CAN_Init();

    commands_init();

    // Envoyer la commande "Allumer servomoteur 1"
    command_t cmd1 = {
        .device_id = 1,
        .command = CMD_SERVO_ON,
        .parameter = 0
    };
    commands_send(&cmd1);

    // Attendre la réponse (via callback HAL_CAN_RxFifo0MsgPendingCallback)
    HAL_Delay(100);

    // Envoyer la commande "Servomoteur 2, position pulse 100"
    command_t cmd2 = {
        .device_id = 2,
        .command = CMD_SERVO_PULSE,
        .parameter = 100
    };
    commands_send(&cmd2);

    while(1) {
        // Boucle principale
    }
}
```

7.4 Exemple 3 : Test en boucle locale (sans matériel CAN)

7.4.1 Scénario

Tester la logique TAGCAN **sans** bus CAN physique.

```
// ==========================================================================
// test_tagcan_local.c
// ==========================================================================

#include "TAGCAN.h"
#include <stdio.h>
#include <string.h>

uint8_t TxData[8] = {0};
uint8_t RxData[8] = {0};

void test_local_loop(void) {
    printf("=== TEST BOUCLE LOCALE TAGCAN ===\n\n");

    // Étape 1: Initialiser
    init_data_container();
    init_tag_manager();

    set_tag("SEQ", 3);
    set_tag("SENSOR_A", 8);
    set_tag("SENSOR_B", 12);

    printf(" Initialisé\n\n");

    // Étape 2: Écrire les données
    printf("Écriture des données...\n");
    set_value("SEQ", 5);
    set_value("SENSOR_A", 42);
    set_value("SENSOR_B", 1000);

    // Afficher le buffer brut
    printf("Buffer TxData: ");
    for(int i = 0; i < 8; i++) {
        printf("%02X ", TxData[i]);
    }
    printf("\n\n");

    // Étape 3: Simuler envoi -> réception
    printf("Simulation envoi/réception...\n");
    can_simulate_send_receive(TxData, RxData);

    // Vérifier
    printf("Buffer RxData: ");
```

```
    for(int i = 0; i < 8; i++) {
        printf("%02X ", RxData[i]);
    }
    printf("\n\n");

    // Étape 4: Lire les données reçues
    printf("Lecture des données reçues...\n");
    memcpy(TxData, RxData, 8);  // Copier pour lecture

    uint32_t seq, sens_a, sens_b;
    get_value("SEQ", &seq);
    get_value("SENSOR_A", &sens_a);
    get_value("SENSOR_B", &sens_b);

    printf("SEQ = %u\n", (uint8_t)seq);
    printf("SENSOR_A = %u\n", (uint8_t)sens_a);
    printf("SENSOR_B = %u\n", (uint16_t)sens_b);

    // Vérifier
    if(seq == 5 && sens_a == 42 && sens_b == 1000) {
        printf("\n TEST RÉUSSI!\n");
    } else {
        printf("\n TEST ÉCHOUÉ!\n");
    }
}

int main(void) {
    test_local_loop();
    return 0;
}
```

Sortie attendue :

```
=== TEST BOUCLE LOCALE TAGCAN ===

 Initialisé

Écriture des données...
Buffer TxData: XX XX XX XX XX XX XX XX

Simulation envoi/réception...
Buffer RxData: XX XX XX XX XX XX XX XX

Lecture des données reçues...
SEQ = 5
SENSOR_A = 42
SENSOR_B = 1000

 TEST RÉUSSI!
```

7.5 Exemple 4 : Debug / Monitoring CAN

7.5.1 Scénario

Logger tous les messages CAN reçus pour du monitoring.

```
// ============================================================================
// can_monitor.h
// ============================================================================

#ifndef CAN_MONITOR_H_
#define CAN_MONITOR_H_

#include <stdint.h>

typedef struct {
    uint32_t id;
    uint8_t dlc;
    uint8_t data[8];
    uint32_t timestamp_ms;
} can_frame_t;

#define CAN_MONITOR_BUFFER_SIZE 100

void can_monitor_init(void);
void can_monitor_log(const can_frame_t *frame);
void can_monitor_dump(void);

#endif


// ============================================================================
// can_monitor.c
// ============================================================================

#include "can_monitor.h"
#include "stm32f1xx_hal.h"
```

```c
#include <stdio.h>
#include <string.h>

static can_frame_t monitor_buffer[CAN_MONITOR_BUFFER_SIZE];
static uint32_t monitor_index = 0;

void can_monitor_init(void) {
    monitor_index = 0;
    memset(monitor_buffer, 0, sizeof(monitor_buffer));
    printf("[Monitor] Initialized\n");
}

void can_monitor_log(const can_frame_t *frame) {
    if(monitor_index >= CAN_MONITOR_BUFFER_SIZE) {
        monitor_index = 0;  // Circulaire
    }

    memcpy(&monitor_buffer[monitor_index], frame, sizeof(can_frame_t));
    monitor_index++;
}

void can_monitor_dump(void) {
    printf("\n=== CAN FRAME LOG ===\n");
    printf("| Index | ID    | DLC | Data                  | Timestamp |\n");
    printf("|-------|-------|-----|----------------------|-----------|\n");

    for(uint32_t i = 0; i < monitor_index && i < CAN_MONITOR_BUFFER_SIZE; i++) {
        const can_frame_t *frame = &monitor_buffer[i];

        printf("| %5u | 0x%03X | %u   | ", i, (unsigned)frame->id, frame->dlc);
        for(int j = 0; j < 8; j++) {
            printf("%02X ", frame->data[j]);
        }
        printf("| %9u |\n", (unsigned)frame->timestamp_ms);
    }
    printf("\n");
}

// ============================================================================
// Utilisation dans le callback CAN
// ============================================================================

void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
    CAN_RxHeaderTypeDef RxHeader;
    uint8_t RxDataLocal[8];

    if(HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxDataLocal) == HAL_OK) {
        can_frame_t frame = {
            .id = RxHeader.StdId,
            .dlc = RxHeader.DLC,
            .timestamp_ms = HAL_GetTick()
        };
        memcpy(frame.data, RxDataLocal, 8);

        can_monitor_log(&frame);
    }
}

// ============================================================================
// main.c
// ============================================================================

int main(void) {
    HAL_Init();
    MX_CAN_Init();

    can_monitor_init();

    // ... votre logique CAN ...

    // Afficher le log après 10 secondes
    HAL_Delay(10000);
    can_monitor_dump();

    while(1) {
        // ...
    }
}
```

7.6 Checklist d'implémentation

- [ ] Inclure `TAGCAN.h`
- [ ] Appeler `init_data_container()`

- [ ] Appeler `init_tag_manager()`
- [ ] Créer les tags via `set_tag()`
- [ ] Configurer le header CAN via `CAN_setup_DF()`
- [ ] Configurer les filtres via `CAN_setup_Filter()`
- [ ] Activer les interruptions
- [ ] Implémenter `HAL_CAN_RxFifo0MsgPendingCallback()`
- [ ] Utiliser `set_value()` pour écrire
- [ ] Utiliser `get_value()` pour lire
- [ ] Envoyer via `HAL_CAN_AddTxMessage()`
- [ ] Recevoir via `HAL_CAN_GetRxMessage()`
- [ ] Tester en boucle locale (optionnel mais recommandé)

Besoin d'aide ? Retour au README.md

Chapter 8

Guide d'Utilisation de TAGCAN

8.1 Aperçu

Ce guide pratique vous aidera à utiliser **TAGCAN**, une bibliothèque de gestion de bus CAN pour systèmes embarqués STM32. La documentation s'appuie sur l'exemple d'implémentation présent dans le dossier `exemple_utilisation/`.

8.2 Structure du guide

- **01_INITIALISATION.md** - Mise en place du système TAGCAN et du bus CAN
- **02_GESTION_TAGS.md** - Création et gestion des tags (variables nommées)
- **03_DONNEES_CAN.md** - Manipulation du buffer de données CAN
- **04_FILTRAGE_CAN.md** - Configuration des filtres CAN
- **05_EXEMPLES_COMPLETS.md** - Exemples pratiques et cas d'usage

8.3 Points clés

8.3.1 Qu'est-ce qu'un tag ?

Un **tag** est une variable nommée associée à une plage de bits dans le buffer CAN. Par exemple :

- `"SEQ"` -> 3 bits (numéro de séquence)
- `"HB"` -> 1 bit (heartbeat/watchdog)
- `"N2OPS"` -> 16 bits (capteur de pression N2O)

8.3.2 Architecture générale

```
-------------------------------
-   Application utilisateur (main.c)          -
-------------------------------
-   TAGCAN (Gestion des tags & données)       -
-  - Tag Manager (init_tag_manager, set_tag) -
-  - Data Container (set_value, get_value)   -
-------------------------------
-   Driver CAN STM32 (HAL)                    -
-------------------------------
```

8.4 Démarrage rapide

```c
// 1. Initialiser TAGCAN
void init_com(void) {
    CAN_init_TAGCAN();      // Initialise le gestionnaire de tags
    CAN_setup_TAG();        // Crée les tags applicatifs
```

```
    CAN_setup_DF(0x123);     // Configure le header CAN
}

// 2. Écrire une valeur dans un tag
set_value("SEQ", 5);         // Écrit 5 dans le tag "SEQ" (3 bits)

// 3. Lire une valeur depuis un tag
uint32_t val;
get_value("HB", &val);       // Lit la valeur du tag "HB"

// 4. Envoyer sur le bus CAN
HAL_CAN_AddTxMessage(&hcan, &TxHeader, TxData, &TxMailbox);
```

8.5 Documentation de référence

Pour plus de détails sur l'API : voir API_REFERENCE.md

8.6 Fichiers sources associés

- `exemple_utilisation/com_utils.c` - Code d'exemple d'intégration
- `exemple_utilisation/com_utils.h` - Headers d'exemple
- `TAGCAN.h` - Point d'entrée principal

Besoin d'aide ? Consultez les exemples complets dans 05_EXEMPLES_COMPLETS.md

Chapter 9

TAGCAN

aide à la gestion CAN

Chapter 10

Data Structure Index

10.1 Data Structures

Here are the data structures with brief descriptions:

TagDef
Structure de definition d'un tag 43
tsfp
Buffer de rendu pour l'affichage de repartition bit-a-bit 44

Chapter 11

File Index

11.1 File List

Here is a list of all files with brief descriptions:

com_utils.c . . . 45
com_utils.h . . . 48
config_value.h . . . 50
gestion_tag.h . . . 53
data_container.h . . . 54
print_val.h . . . 57
tag_manager.h . . . 59
CAN_utils.c . . . 60
config_value.c . . . 60
main.c . . . 62
data_container.c . . . 63
print_val.c . . . 67
tag_manager.c . . . 69
TAGCAN.h . . . 72
test_print_repartition_bit.c . . . 73
test_safty_func.c . . . 73
test_safty_func.h . . . 73
test_send_receve.c . . . 73
test_send_receve.h . . . 73
test_set_tag.c . . . 73
test_set_tag.h . . . 73
test_utils.c . . . 73
test_utils.h . . . 74
version.h . . . 75

Chapter 12

Data Structure Documentation

12.1 TagDef Struct Reference

Structure de definition d'un tag.
`#include <tag_manager.h>`

Data Fields

- char name [MAX_TAG_NAME_LEN]
- uint8_t n_bits
- int8_t byte_idx_a
- int8_t byte_idx_b
- uint8_t bit_pos_a
- uint8_t bit_pos_b

12.1.1 Detailed Description

Structure de definition d'un tag.

12.1.2 Field Documentation

12.1.2.1 bit_pos_a

`uint8_t bit_pos_a`
Position de debut dans le byte A (de gauche a droite).

12.1.2.2 bit_pos_b

`uint8_t bit_pos_b`
Position de debut dans le byte B pour les tags > 8 bits, 0 sinon.

12.1.2.3 byte_idx_a

`int8_t byte_idx_a`
Index du byte de la partie A (0..7), -1 si non utilise.

12.1.2.4 byte_idx_b

`int8_t byte_idx_b`
Index du byte de la partie B (tags > 8 bits), sinon -1.

12.1.2.5 n_bits

`uint8_t n_bits`
Taille en bits (implementation actuelle: 1..16, 0 si slot inutilise).

12.1.2.6 name

`char name[MAX_TAG_NAME_LEN]`
Nom logique du tag. Doit etre unique.
The documentation for this struct was generated from the following file:

- tag_manager.h

12.2 tsfp Struct Reference

Buffer de rendu pour l'affichage de repartition bit-a-bit.
`#include <print_val.h>`

Data Fields

- char bi [8]

12.2.1 Detailed Description

Buffer de rendu pour l'affichage de repartition bit-a-bit.

12.2.2 Field Documentation

12.2.2.1 bi

`char bi[8]`
The documentation for this struct was generated from the following file:

- print_val.h

Chapter 13

File Documentation

13.1 API_REFERENCE.md File Reference

13.2 README.md File Reference

13.3 README.md File Reference

13.4 README.md File Reference

13.5 com_utils.c File Reference

```
#include "com_utils.h"
#include "../../TAGCAN/include/sub_include/data_container.h"
```

Functions

- void init_com (void)
- void CAN_init_TAGCAN (void)
- void CAN_setup_DF (int32_t std_id)
- void CAN_setup_TAG (void)
- CAN_FilterTypeDef * CAN_setup_Filter (int32_t minimum_id, int32_t maximum_id, int filterBank)
- void main_loop (void)
- void update_TagData (void)
- void CAN_send_msg (void)
- void CAN_change_val (void)
- void update_val (const char *nom_du_tag, int32_t val)
- void update_all_val_send (void)
- void update_all_val_receve (void)
- void HAL_CAN_RxFifo0MsgPendingCallback (CAN_HandleTypeDef *hcan)

Variables

- CAN_TxHeaderTypeDef TxHeader
- CAN_RxHeaderTypeDef RxHeader
- uint8_t RxData [8]
- int8_t val_SEQ = 0
- int8_t val_HB = 0
- int8_t val_CI = 0
- int8_t val_AB = 0
- int8_t val_EDC = 0

- int16_t val_N2OPS = 0
- int16_t val_TLC = 0
- int8_t val_N2ODVS = 0
- int8_t val_N2OIV = 0
- int8_t val_N2OMV = 0
- int16_t val_TIME = 0
- int8_t out_Rx [8] = {0,0,0,0,0,0,0,0}
- int datacheck = 0
- int8_t TxData_moteur [8] = {0,0,0,0,0,0,0,0}
- int8_t RxData_moteur [8] = {0,0,0,0,0,0,0,0}
- int32_t TxMailbox

13.5.1 Function Documentation

13.5.1.1 CAN_change_val()

```
void CAN_change_val (
            void  )
```

13.5.1.2 CAN_init_TAGCAN()

```
void CAN_init_TAGCAN (
            void  )
```

13.5.1.3 CAN_send_msg()

```
void CAN_send_msg (
            void  )
```

13.5.1.4 CAN_setup_DF()

```
void CAN_setup_DF (
            int32_t std_id )
```

13.5.1.5 CAN_setup_Filter()

```
CAN_FilterTypeDef * CAN_setup_Filter (
            int32_t minimum_id,
            int32_t maximum_id,
            int filterBank )
```

13.5.1.6 CAN_setup_TAG()

```
void CAN_setup_TAG (
            void  )
```

13.5.1.7 HAL_CAN_RxFifo0MsgPendingCallback()

```
void HAL_CAN_RxFifo0MsgPendingCallback (
            CAN_HandleTypeDef * hcan )
```

13.5.1.8 init_com()

```
void init_com (
            void  )
```

- initialise tout les fonction de communication. (point d'appel unique: main) */

13.5.1.9 main_loop()

```
void main_loop (
            void  )
```

13.5.1.10 update_all_val_receve()

```
void update_all_val_receve (
            void  )
```

13.5.1.11 update_all_val_send()

```
void update_all_val_send (
            void  )
```

13.5.1.12 update_TagData()

```
void update_TagData (
            void  )
```

13.5.1.13 update_val()

```
void update_val (
            const char * nom_du_tag,
            int32_t val )
```

13.5.2 Variable Documentation

13.5.2.1 datacheck

```
int datacheck = 0
```

13.5.2.2 out_Rx

```
int8_t out_Rx[8] = {0,0,0,0,0,0,0,0}
```

13.5.2.3 RxData

```
uint8_t RxData[8]
```

13.5.2.4 RxData_moteur

```
int8_t RxData_moteur[8] = {0,0,0,0,0,0,0,0}
```

13.5.2.5 RxHeader

```
CAN_RxHeaderTypeDef RxHeader
```

13.5.2.6 TxData_moteur

```
int8_t TxData_moteur[8] = {0,0,0,0,0,0,0,0}
```

13.5.2.7 TxHeader

```
CAN_TxHeaderTypeDef TxHeader
```

13.5.2.8 TxMailbox

```
int32_t TxMailbox
```

13.5.2.9 val_AB

```
int8_t val_AB = 0
```

13.5.2.10 val_CI

```
int8_t val_CI = 0
```

13.5.2.11 val_EDC

```
int8_t val_EDC = 0
```

13.5.2.12 val_HB

```
int8_t val_HB = 0
```

13.5.2.13 val_N2ODVS

```
int8_t val_N2ODVS = 0
```

13.5.2.14 val_N2OIV

```
int8_t val_N2OIV = 0
```

13.5.2.15 val_N2OMV

```
int8_t val_N2OMV = 0
```

13.5.2.16 val_N2OPS

```
int16_t val_N2OPS = 0
```

13.5.2.17 val_SEQ

```
int8_t val_SEQ = 0
```

13.5.2.18 val_TIME

```
int16_t val_TIME = 0
```

13.5.2.19 val_TLC

```
int16_t val_TLC = 0
```

13.6 com_utils.h File Reference

```
#include "../../TAGCAN/TAGCAN.h"
#include "../../TAGCAN/include/sub_include/data_container.h"
#include <cmsis_gcc.h>
#include <stm32f103xb.h>
#include <stm32f1xx.h>
#include <stm32f1xx_hal_can.h>
#include <stm32f1xx_hal_def.h>
#include <stm32f1xx_hal_flash.h>
#include <stm32f1xx_hal_rcc.h>
#include <stm32f1xx_hal_rcc_ex.h>
#include <sys/_stdint.h>
```

Functions

- void init_com (void)

Variables

- uint8_t TxData [8]
- uint8_t RxData [8]
- CAN_TxHeaderTypeDef TxHeader
- CAN_RxHeaderTypeDef RxHeader
- CAN_HandleTypeDef hcan
- int32_t TxMailbox
- int8_t val_SEQ
- int8_t val_HB
- int8_t val_CI
- int8_t val_AB
- int8_t val_EDC
- int8_t val_N2ODVS
- int8_t val_N2OIV
- int8_t val_N2OMV
- int16_t val_N2OPS
- int16_t val_TIME
- int16_t val_TLC

13.6.1 Function Documentation

13.6.1.1 init_com()

```
void init_com (
            void  )
```

- initialise tout les fonction de communication. (point d'appel unique: main) */

13.6.2 Variable Documentation

13.6.2.1 hcan

```
CAN_HandleTypeDef hcan  [extern]
```

13.6.2.2 RxData

```
uint8_t RxData[8]  [extern]
```

13.6.2.3 RxHeader

```
CAN_RxHeaderTypeDef RxHeader  [extern]
```

13.6.2.4 TxData

```
uint8_t TxData[8]  [extern]
```

13.6.2.5 TxHeader

```
CAN_TxHeaderTypeDef TxHeader  [extern]
```

13.6.2.6 TxMailbox

```
int32_t TxMailbox  [extern]
```

13.6.2.7 val_AB

```
int8_t val_AB [extern]
```

13.6.2.8 val_CI

```
int8_t val_CI [extern]
```

13.6.2.9 val_EDC

```
int8_t val_EDC [extern]
```

13.6.2.10 val_HB

```
int8_t val_HB [extern]
```

13.6.2.11 val_N2ODVS

```
int8_t val_N2ODVS [extern]
```

13.6.2.12 val_N2OIV

```
int8_t val_N2OIV [extern]
```

13.6.2.13 val_N2OMV

```
int8_t val_N2OMV [extern]
```

13.6.2.14 val_N2OPS

```
int16_t val_N2OPS [extern]
```

13.6.2.15 val_SEQ

```
int8_t val_SEQ [extern]
```

13.6.2.16 val_TIME

```
int16_t val_TIME [extern]
```

13.6.2.17 val_TLC

```
int16_t val_TLC [extern]
```

13.7 01_INITIALISATION.md File Reference

13.8 02_GESTION_TAGS.md File Reference

13.9 03_DONNEES_CAN.md File Reference

13.10 04_FILTRAGE_CAN.md File Reference

13.11 05_EXEMPLES_COMPLETS.md File Reference

13.12 config_value.h File Reference

Macros

- #define MAX_TAGS 32

- #define MAX_TAG_NAME_LEN 16
- #define N_BYTES 8
- #define MAX_N_GT 3

Enumerations

- enum CAN_TG_STATUE { CAN_TG_ERROR_NOT_ENOUNG_SPACE_ = -2 , CAN_TG_SUCCESS = 1 , CAN_TG_ERROR_TAG_NOT_FOUND_ = -3 , CAN_TG_ERROR_TAG_NAME_TO_BIGS_ = -4 , CAN_TG_ERROR_TOO_MAY_TAGS_ = -5 , CAN_TG_ERROR_SIZE_VALUE_INVALIDE_ = -6 , CAN_TG_ERROR_NAME_ALREADY_USE_ = -7 , CAN_TG_ERROR_SUBDIVISION_FAIL_ = -8 , CAN_TG_ERROR_TAG_NAME_INVALIDE_ =-9 , CAN_TG_ERROR_NULL_POINTOR_ = -2 , CAN_TG_ERROR_TRY_TO_FI = -21 , CAN_TG_ERROR_INTERNAL_CORUPTION_ = -30 , CAN_TG_ERROR_VALUE_TO_BIG = -10 }

  Codes de retour du module TAGCAN.
- enum CAN_MODE { RECEVE_ONLY , SEND_ONLY , SEND_AND_RECEVE }

  modes d'utilisation CAN (envoi/reception).

Variables

- const int CAN_TG_ERROR_INTERNAL_CORUPTION
- const int CAN_TG_ERROR_TAG_NAME_TO_BIGS
- const int CAN_TG_ERROR_TAG_NAME_INVALIDE
- const int CAN_TG_ERROR_TAG_NOT_FOUND
- const int CAN_TG_ERROR_NOT_ENOUNG_SPACE
- const int CAN_TG_ERROR_NULL_POINTOR
- const int CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER
- const int CAN_TG_ERROR_SIZE_VALUE_INVALIDE
- const int CAN_TG_ERROR_SUBDIVISION_FAIL
- const int CAN_TG_ERROR_TOO_MAY_TAGS
- const int CAN_TG_ERROR_NAME_ALREADY_USE
- const int SUCCES_TO_SET
- const int SUCCES

13.12.1 Macro Definition Documentation

13.12.1.1 MAX_N_GT

```
#define MAX_N_GT 3
```

Nombre maximal de gestionnaires (reserve/extension).

13.12.1.2 MAX_TAG_NAME_LEN

```
#define MAX_TAG_NAME_LEN 16
```

Longueur maximale d'un nom de tag (incluant le '\0').

13.12.1.3 MAX_TAGS

```
#define MAX_TAGS 32
```

Nombre maximal de tags geres simultanement.

13.12.1.4 N_BYTES

```
#define N_BYTES 8
```

Taille du payload CAN en octets.

13.12.2 Enumeration Type Documentation

13.12.2.1 CAN_MODE

`enum CAN_MODE`

modes d'utilisation CAN (envoi/reception).

Enumerator

RECEVE_ONLY	Réception seule: les mises à jour entrantes peuvent modifier la valeur; l'écriture locale est bloquée.
SEND_ONLY	Émission seule: la valeur locale est mise, les mises à jour entrantes ne la modifient pas.
SEND_AND_RECEVE	Emission + reception: écriture locale et mises à jour entrantes toutes deux autorisées.

13.12.2.2 CAN_TG_STATUE

`enum CAN_TG_STATUE`

Codes de retour du module TAGCAN.

Enumerator

CAN_TG_ERROR_NOT_ENOUNG_SPACE_	
CAN_TG_SUCCESS	
CAN_TG_ERROR_TAG_NOT_FOUND_	
CAN_TG_ERROR_TAG_NAME_TO_BIGS_	
CAN_TG_ERROR_TOO_MAY_TAGS_	
CAN_TG_ERROR_SIZE_VALUE_INVALIDE_	
CAN_TG_ERROR_NAME_ALREADY_USE_	
CAN_TG_ERROR_SUBDIVISION_FAIL_	
CAN_TG_ERROR_TAG_NAME_INVALIDE_	
CAN_TG_ERROR_NULL_POINTOR_	
CAN_TG_ERROR_TRY_TO_FIND_END_↩ CARACTER_	
CAN_TG_ERROR_INTERNAL_CORUPTION_	
CAN_TG_ERROR_VALUE_TO_BIG	

13.12.3 Variable Documentation

13.12.3.1 CAN_TG_ERROR_INTERNAL_CORUPTION

`const int CAN_TG_ERROR_INTERNAL_CORUPTION [extern]`

Code d'erreur: corruption interne detectee.

13.12.3.2 CAN_TG_ERROR_NAME_ALREADY_USE

`const int CAN_TG_ERROR_NAME_ALREADY_USE [extern]`

Code d'erreur: nom de tag deja utilise.

13.12.3.3 CAN_TG_ERROR_NOT_ENOUNG_SPACE

`const int CAN_TG_ERROR_NOT_ENOUNG_SPACE [extern]`

Code d'erreur: espace memoire insuffisant.

13.12.3.4 CAN_TG_ERROR_NULL_POINTOR

`const int CAN_TG_ERROR_NULL_POINTOR [extern]`
Code d'erreur: pointeur NULL recu.

13.12.3.5 CAN_TG_ERROR_SIZE_VALUE_INVALIDE

`const int CAN_TG_ERROR_SIZE_VALUE_INVALIDE [extern]`
Code d'erreur: taille de valeur invalide.

13.12.3.6 CAN_TG_ERROR_SUBDIVISION_FAIL

`const int CAN_TG_ERROR_SUBDIVISION_FAIL [extern]`
Code d'erreur: echec de subdivision/allocation.

13.12.3.7 CAN_TG_ERROR_TAG_NAME_INVALIDE

`const int CAN_TG_ERROR_TAG_NAME_INVALIDE [extern]`
Code d'erreur: nom de tag invalide.

13.12.3.8 CAN_TG_ERROR_TAG_NAME_TO_BIGS

`const int CAN_TG_ERROR_TAG_NAME_TO_BIGS [extern]`
Code d'erreur: nom de tag trop long.

13.12.3.9 CAN_TG_ERROR_TAG_NOT_FOUND

`const int CAN_TG_ERROR_TAG_NOT_FOUND [extern]`
Code d'erreur: tag introuvable.

13.12.3.10 CAN_TG_ERROR_TOO_MAY_TAGS

`const int CAN_TG_ERROR_TOO_MAY_TAGS [extern]`
Code d'erreur: trop de tags declares.

13.12.3.11 CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER

`const int CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER [extern]`
Code d'erreur: fin de chaine non trouvee.

13.12.3.12 SUCCES

`const int SUCCES [extern]`
Code de succes generique historique.

13.12.3.13 SUCCES_TO_SET

`const int SUCCES_TO_SET [extern]`
Code de succes historique pour set_tag/set_value.

13.13 gestion_tag.h File Reference

Functions

- void init_tag_manager (void)

  Initialise/reinitialise le gestionnaire de tags.
- int set_tag (const char ∗tag_name, uint8_t n_bit)

  créé un nouveau tag.

13.13.1 Function Documentation

13.13.1.1 init_tag_manager()

```
void init_tag_manager (
            void  )
```

Initialise/reinitialise le gestionnaire de tags.

Note

Apres cet appel, l'ensemble des tags doit etre redefini via `set_tag`.

13.13.1.2 set_tag()

```
int set_tag (
            const char * tag_name,
            uint8_t siz )
```

créé un nouveau tag.

Parameters

tag_name	le nom (id) relié a la valeur.
n_bit	La taille de la valeur (implémentation actuelle: 1 à 16 bits).

Returns

un code de succès ou d'erreur.

créé un nouveau tag.

Parameters

tag_name	Nom logique du tag.
siz	Taille de la valeur en bits (implementation actuelle: 1..16).

Returns

Code de succes ou code d'erreur CAN_TG_STATUE.

13.14 data_container.h File Reference

```
#include <stdint.h>
#include "tag_manager.h"
```

Functions

- void init_data_container (void)

  Reinitialise le payload TxData a zero.
- int set_value (const char ∗tag_name, uint32_t value)

  Ecrit une valeur dans TxData a partir d'un tag.
- int get_value (const char ∗tag_name, uint32_t ∗out_value)

  Lit la valeur associee a un tag.
- uint8_t prep_mask (uint8_t p_bit, uint8_t n_bit)

  [FONCTION INTERNE] Prepare un masque binaire pour set/get.
- void can_simulate_send_receive (uint8_t ∗tx_buffer, uint8_t ∗rx_buffer)

  Simule un envoi/reception CAN par copie buffer-a-buffer.

- void set_val_TxData (uint8_t *TxDataArray)

  Copie un tableau externe dans le buffer global TxData.

Variables

- uint8_t TxData [N_BYTES]

13.14.1 Function Documentation

13.14.1.1 can_simulate_send_receive()

```
void can_simulate_send_receive (
            uint8_t * tx_buffer,
            uint8_t * rx_buffer )
```

Simule un envoi/reception CAN par copie buffer-a-buffer.

Parameters

tx_buffer	Buffer source (emission).
rx_buffer	Buffer destination (reception).

Simule un envoi/reception CAN par copie buffer-a-buffer.
Copie simplement tx_buffer dans rx_buffer.

Parameters

tx_buffer	Buffer d'emission.
rx_buffer	Buffer de reception.

13.14.1.2 get_value()

```
int get_value (
            const char * tag_name,
            uint32_t * out_value )
```

Lit la valeur associee a un tag.

Parameters

tag_name	Nom du tag a lire.
out_value	Adresse de sortie qui recoit la valeur extraite.

Returns

CAN_TG_SUCCESS en cas de succes, sinon un code d'erreur negatif.

Copie un buffer externe dans TxData.

Parameters

TxDataArray	Tableau source de N_BYTES octets.

Recompose un entier 16 bits depuis deux octets.

Parameters

ba	Octet de poids faible.

Parameters

bb	Octet de poids fort.

Returns

Valeur 16 bits combinee.

13.14.1.3 init_data_container()

```
void init_data_container (
            void  )
```

Reinitialise le payload TxData a zero.
Si reexecutee, la fonction efface les donnees precedemment ecrites.

13.14.1.4 prep_mask()

```
uint8_t prep_mask (
            uint8_t p_bit,
            uint8_t n_bit )
```

[FONCTION INTERNE] Prepare un masque binaire pour set/get.
Utilise principalement pour les valeurs strictement inferieures a 8 bits.

Parameters

p_bit	Position du premier bit (de gauche a droite).
n_bit	Nombre de bits de la valeur.

Returns

Masque binaire aligne sur la position cible.

13.14.1.5 set_val_TxData()

```
void set_val_TxData (
            uint8_t * TxDataArray )
```

Copie un tableau externe dans le buffer global TxData.

Parameters

TxDataArray	Tableau source de N_BYTES octets.

13.14.1.6 set_value()

```
int set_value (
            const char * tag_name,
            uint32_t value )
```

Ecrit une valeur dans TxData a partir d'un tag.

Warning

La valeur doit tenir dans la taille du tag.

En cas de depassement, l'implementation retourne CAN_TG_ERROR_VALUE_TO_BIG.

Parameters

tag_name	Nom du tag cible.
value	Nouvelle valeur a ecrire.

Returns

Code de succes ou code d'erreur.

13.14.2 Variable Documentation

13.14.2.1 TxData

```
uint8_t TxData[N_BYTES]  [extern]
```

Buffer de donnees CAN (payload de 8 octets).

13.15 print_val.h File Reference

```
#include <stdio.h>
#include <stdint.h>
```

Data Structures

- struct tsfp

  Buffer de rendu pour l'affichage de repartition bit-a-bit.

Functions

- void print_bin_8 (uint8_t val)

  Affiche la representation binaire (8 bits) d'une valeur.
- void print_all_data_bin (void)

  Affiche tout le payload TxData en binaire.
- void print_all_with_title (const char *title)

  Affiche TxData en binaire avec un titre en prefixe.
- void print_all_tag_set (void)

  Affiche la table des tags declares et leurs metadonnees.
- void print_all_data (void)

  Affiche toutes les valeurs logiques relues via get_value.
- void print_repartition_bit (void)

  Affiche la repartition des bits utilises dans la trame CAN.

Variables

- tsfp gbc2 [9]

13.15.1 Function Documentation

13.15.1.1 print_all_data()

```
void print_all_data (
            void  )
```

Affiche toutes les valeurs logiques relues via get_value.
Affiche toutes les valeurs logiques relues via get_value.
Chaque valeur est relue via get_value puis imprimee.

13.15.1.2 print_all_data_bin()

```
void print_all_data_bin (
            void  )
```

Affiche tout le payload TxData en binaire.
Chaque octet est affiche sur 8 bits, dans l'ordre du buffer.
Affiche tout le payload TxData en binaire.
Prints the binary representation of each byte in the `TxData` array.

13.15.1.3 print_all_tag_set()

```
void print_all_tag_set (
            void  )
```

Affiche la table des tags declares et leurs metadonnees.
Affiche la table des tags declares et leurs metadonnees.

13.15.1.4 print_all_with_title()

```
void print_all_with_title (
            const char * title )
```

Affiche TxData en binaire avec un titre en prefixe.

Parameters

title	Texte affiche avant la ligne binaire.

Affiche TxData en binaire avec un titre en prefixe.

Parameters

title	Libelle affiche avant les octets.

13.15.1.5 print_bin_8()

```
void print_bin_8 (
            uint8_t val )
```

Affiche la representation binaire (8 bits) d'une valeur.

Parameters

val	Valeur a afficher en binaire sur 8 bits.

Affiche la representation binaire (8 bits) d'une valeur.

Parameters

val	The `val` [from 0 to 255]. The function prints the binary representation of this value by iterating over each bit.

13.15.1.6 print_repartition_bit()

```
void print_repartition_bit (
            void  )
```

Affiche la repartition des bits utilises dans la trame CAN.
Affiche la repartition des bits utilises dans la trame CAN.

13.15.2 Variable Documentation

13.15.2.1 gbc2

```
tsfp gbc2[9] [extern]
```

Espace global de rendu utilise par les fonctions d'impression de repartition.

13.16 tag_manager.h File Reference

```
#include "../config_value.h"
#include <stdint.h>
#include <stdbool.h>
```

Data Structures

- struct TagDef

  Structure de definition d'un tag.

Functions

- const TagDef ∗ get_tag_def (const char ∗tag_name)

  Recherche un tag et retourne sa definition.

Variables

- TagDef g_tags [MAX_TAGS]
- uint8_t g_tag_count
- uint8_t g_bytes_used [N_BYTES]

13.16.1 Function Documentation

13.16.1.1 get_tag_def()

```
const TagDef * get_tag_def (
            const char * tag_name )
```

Recherche un tag et retourne sa definition.

Warning

API surtout utilisee en interne par le module de valeurs.

Parameters

tag_name	Nom du tag a chercher.

Returns

Pointeur vers le tag si trouve, sinon NULL.

Recherche un tag et retourne sa definition.

Parameters

tag_name	Nom du tag recherche.

Returns

Pointeur sur TagDef si trouve, sinon NULL.

13.16.2 Variable Documentation

13.16.2.1 g_bytes_used

```
uint8_t g_bytes_used[N_BYTES] [extern]
```

Occupation courante en bits pour chaque byte du payload.

13.16.2.2 g_tag_count

```
uint8_t g_tag_count [extern]
```

Nombre de tags actifs (max: MAX_TAGS).

13.16.2.3 g_tags

```
TagDef g_tags[MAX_TAGS] [extern]
```

Table des tags (actifs et inactifs).

13.17 CAN_utils.c File Reference

13.18 config_value.c File Reference

```
#include "../include/config_value.h"
```

Variables

- const int CAN_TG_ERROR_NOT_ENOUNG_SPACE = -2
- const int CAN_TG_ERROR_TAG_NOT_FOUND = -3
- const int CAN_TG_ERROR_TAG_NAME_TO_BIGS = -4
- const int CAN_TG_ERROR_TOO_MAY_TAGS = -5
- const int CAN_TG_ERROR_SIZE_VALUE_INVALIDE = -6
- const int CAN_TG_ERROR_NAME_ALREADY_USE = -7
- const int CAN_TG_ERROR_SUBDIVISION_FAIL = -8
- const int CAN_TG_ERROR_TAG_NAME_INVALIDE =-9
- const int CAN_TG_ERROR_NULL_POINTOR = -20
- const int CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER = -21
- const int CAN_TG_ERROR_INTERNAL_CORUPTION = -30
- const int SUCCES_TO_SET = 1
- const int SUCCES = 1

13.18.1 Variable Documentation

13.18.1.1 CAN_TG_ERROR_INTERNAL_CORUPTION

```
const int CAN_TG_ERROR_INTERNAL_CORUPTION = -30
```

Code d'erreur: corruption interne detectee.

13.18.1.2 CAN_TG_ERROR_NAME_ALREADY_USE

```
const int CAN_TG_ERROR_NAME_ALREADY_USE = -7
```

Code d'erreur: nom de tag deja utilise.

13.18.1.3 CAN_TG_ERROR_NOT_ENOUNG_SPACE

`const int CAN_TG_ERROR_NOT_ENOUNG_SPACE = -2`
Code d'erreur: espace memoire insuffisant.

13.18.1.4 CAN_TG_ERROR_NULL_POINTOR

`const int CAN_TG_ERROR_NULL_POINTOR = -20`
Code d'erreur: pointeur NULL recu.

13.18.1.5 CAN_TG_ERROR_SIZE_VALUE_INVALIDE

`const int CAN_TG_ERROR_SIZE_VALUE_INVALIDE = -6`
Code d'erreur: taille de valeur invalide.

13.18.1.6 CAN_TG_ERROR_SUBDIVISION_FAIL

`const int CAN_TG_ERROR_SUBDIVISION_FAIL = -8`
Code d'erreur: echec de subdivision/allocation.

13.18.1.7 CAN_TG_ERROR_TAG_NAME_INVALIDE

`const int CAN_TG_ERROR_TAG_NAME_INVALIDE =-9`
Code d'erreur: nom de tag invalide.

13.18.1.8 CAN_TG_ERROR_TAG_NAME_TO_BIGS

`const int CAN_TG_ERROR_TAG_NAME_TO_BIGS = -4`
Code d'erreur: nom de tag trop long.

13.18.1.9 CAN_TG_ERROR_TAG_NOT_FOUND

`const int CAN_TG_ERROR_TAG_NOT_FOUND = -3`
Code d'erreur: tag introuvable.

13.18.1.10 CAN_TG_ERROR_TOO_MAY_TAGS

`const int CAN_TG_ERROR_TOO_MAY_TAGS = -5`
Code d'erreur: trop de tags declares.

13.18.1.11 CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER

`const int CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER = -21`
Code d'erreur: fin de chaine non trouvee.

13.18.1.12 SUCCES

`const int SUCCES = 1`
Code de succes generique historique.

13.18.1.13 SUCCES_TO_SET

`const int SUCCES_TO_SET = 1`
Code de succes historique pour set_tag/set_value.

13.19 main.c File Reference

```
#include <stdio.h>
#include <assert.h>
#include <stdbool.h>
#include "../include/config_value.h"
#include "../include/sub_include/data_container.h"
#include "../include/gestion_tag.h"
#include "../include/sub_include/print_val.h"
```

Functions

- void TEST (const char *name, const bool condition)

  Affiche un resultat de test avec format uniforme.
- void TEST_EQ_INT (const char *name, const int val_1, const int val_2)

  Affiche un test d'egalite entre deux entiers.
- void test_print (void)

  Execute un scenario de demonstration des fonctions d'impression.
- void somme_other_test (void)

  Scenario de test manuel des operations de masque/shift.
- void run_all_tests (void)

  Execute les tests unitaires du module. Les resultats sont affiches dans le terminal.
- int main (void)

Variables

- int passed_tests = 0
- int total_tests = 0

13.19.1 Function Documentation

13.19.1.1 main()

```
int main (
            void  )
```

13.19.1.2 run_all_tests()

```
void run_all_tests (
            void  )
```

Execute les tests unitaires du module. Les resultats sont affiches dans le terminal.

13.19.1.3 somme_other_test()

```
void somme_other_test (
            void  )
```

Scenario de test manuel des operations de masque/shift.

13.19.1.4 TEST()

```
void TEST (
            const char * name,
            const bool condition )
```

Affiche un resultat de test avec format uniforme.

Parameters

name	Le nom du test (peut aussi servir de description courte).

Parameters

condition	Resultat du test (true si reussi, false sinon).

13.19.1.5 TEST_EQ_INT()

```
void TEST_EQ_INT (
            const char * name,
            const int val_1,
            const int val_2 )
```

Affiche un test d'egalite entre deux entiers.

Parameters

name	Meme usage que TEST.
val↩ _1	Premiere valeur a comparer.
val↩ _2	Deuxieme valeur a comparer.

13.19.1.6 test_print()

```
void test_print (
            void )
```

Execute un scenario de demonstration des fonctions d'impression.
Initialise des tags/valeurs puis affiche metadonnees et repartition.

13.19.2 Variable Documentation

13.19.2.1 passed_tests

```
int passed_tests = 0
```

le nombre total de test réussi

13.19.2.2 total_tests

```
int total_tests = 0
```

le nombre total de test effectué

13.20 data_container.c File Reference

```
#include "../../include/sub_include/data_container.h"
#include "../../include/config_value.h"
#include "../../include/sub_include/print_val.h"
#include <stdio.h>
```

Functions

- void init_data_container (void)

  Reinitialise le payload TxData a zero.
- void set_val_TxData (uint8_t ∗TxDataArray)

  Copie un tableau externe dans le buffer global TxData.
- uint8_t prep_mask (uint8_t p_bit, uint8_t n_bit)

  [FONCTION INTERNE] Prepare un masque binaire pour set/get.

- CAN_TG_STATUE set_value (const char ∗tag_name, uint32_t value)

  Ecrit une valeur dans TxData a partir d'un tag.
- uint16_t convert_8_to_16bit (uint8_t ba, uint8_t bb)
- int get_value (const char ∗tag_name, uint32_t ∗out_value)

  Lit la valeur associee a un tag.
- void can_simulate_send_receive (uint8_t ∗tx_buffer, uint8_t ∗rx_buffer)

  Simule une transmission CAN locale.

Variables

- uint8_t TxData [N_BYTES]

13.20.1 Function Documentation

13.20.1.1 can_simulate_send_receive()

```
void can_simulate_send_receive (
            uint8_t * tx_buffer,
            uint8_t * rx_buffer )
```

Simule une transmission CAN locale.
Simule un envoi/reception CAN par copie buffer-a-buffer.
Copie simplement tx_buffer dans rx_buffer.

Parameters

tx_buffer	Buffer d'emission.
rx_buffer	Buffer de reception.

13.20.1.2 convert_8_to_16bit()

```
uint16_t convert_8_to_16bit (
            uint8_t ba,
            uint8_t bb )
```

13.20.1.3 get_value()

```
int get_value (
            const char * tag_name,
            uint32_t * out_value )
```

Lit la valeur associee a un tag.

Parameters

tag_name	Nom du tag a lire.
out_value	Adresse de sortie qui recoit la valeur extraite.

Returns

CAN_TG_SUCCESS en cas de succes, sinon un code d'erreur negatif.

Copie un buffer externe dans TxData.

Parameters

TxDataArray	Tableau source de N_BYTES octets.

Recompose un entier 16 bits depuis deux octets.

Parameters

ba	Octet de poids faible.
bb	Octet de poids fort.

Returns

Valeur 16 bits combinee.

13.20.1.4 init_data_container()

```
void init_data_container (
            void  )
```

Reinitialise le payload TxData a zero.
Si reexecutee, la fonction efface les donnees precedemment ecrites.

13.20.1.5 prep_mask()

```
uint8_t prep_mask (
            uint8_t p_bit,
            uint8_t n_bit )
```

[FONCTION INTERNE] Prepare un masque binaire pour set/get.
Utilise principalement pour les valeurs strictement inferieures a 8 bits.

Parameters

p_bit	Position du premier bit (de gauche a droite).
n_bit	Nombre de bits de la valeur.

Returns

Masque binaire aligne sur la position cible.

13.20.1.6 set_val_TxData()

```
void set_val_TxData (
            uint8_t * TxDataArray )
```

Copie un tableau externe dans le buffer global TxData.

Parameters

TxDataArray	Tableau source de N_BYTES octets.

13.20.1.7 set_value()

```
CAN_TG_STATUE set_value (
            const char * tag_name,
            uint32_t value )
```

Ecrit une valeur dans TxData a partir d'un tag.

Warning

La valeur doit tenir dans la taille du tag.

En cas de depassement, l'implementation retourne CAN_TG_ERROR_VALUE_TO_BIG.

Parameters

tag_name	Nom du tag cible.
value	Nouvelle valeur a ecrire.

Returns

Code de succes ou code d'erreur.

13.20.2 Variable Documentation

13.20.2.1 TxData

`uint8_t TxData[N_BYTES]`
Buffer de donnees CAN (payload de 8 octets).

13.21 print_val.c File Reference

```
#include "../../include/sub_include/print_val.h"
#include "../../include/sub_include/tag_manager.h"
#include "../../include/sub_include/data_container.h"
#include <stdio.h>
#include <stdint.h>
#include <string.h>
```

Functions

- void print_all_tag_set (void)

  Affiche la liste complete des tags et leurs metadonnees.
- void print_all_data (void)

  Affiche toutes les valeurs logiques des tags.
- void modif_vt (int8_t bia, int8_t bib, uint8_t bpa, uint8_t bpb, uint8_t nb, char ∗v1, char ∗v2, char ∗v3, char ∗v4, char ∗v5, char ∗v6, char ∗v7, char ∗v8)

  Affiche la repartition des tags dans les bytes de la trame CAN.
- void print_repartition_bit (void)

  Affiche une vue textuelle de l'occupation des bits de la trame.
- void print_bin_8 (uint8_t val)

  Prints the binary representation of an 8-bit unsigned integer.
- void print_all_data_bin (void)

  Prints the binary representation of each byte in `TxData`.
- void print_all_with_title (const char ∗title)

  Affiche TxData en binaire avec un titre.

Variables

- tsfp gbc2 [9]

13.21.1 Function Documentation

13.21.1.1 modif_vt()

```
void modif_vt (
            int8_t bia,
            int8_t bib,
            uint8_t bpa,
```

```
            uint8_t bpb,
            uint8_t nb,
            char * v1,
            char * v2,
            char * v3,
            char * v4,
            char * v5,
            char * v6,
            char * v7,
            char * v8 )
```

Affiche la repartition des tags dans les bytes de la trame CAN.
[INTERNE] Point d'extension pour la construction de vues de repartition.

Warning

Fonction actuellement inachevee/conservee pour evolution future.

13.21.1.2 print_all_data()

```
void print_all_data (
            void  )
```

Affiche toutes les valeurs logiques des tags.
Affiche toutes les valeurs logiques relues via get_value.
Chaque valeur est relue via get_value puis imprimee.

13.21.1.3 print_all_data_bin()

```
void print_all_data_bin (
            void  )
```

Prints the binary representation of each byte in `TxData`.
Affiche tout le payload TxData en binaire.
Prints the binary representation of each byte in the `TxData` array.

13.21.1.4 print_all_tag_set()

```
void print_all_tag_set (
            void  )
```

Affiche la liste complete des tags et leurs metadonnees.
Affiche la table des tags declares et leurs metadonnees.

13.21.1.5 print_all_with_title()

```
void print_all_with_title (
            const char * title )
```

Affiche TxData en binaire avec un titre.
Affiche TxData en binaire avec un titre en prefixe.

Parameters

title	Libelle affiche avant les octets.

13.21.1.6 print_bin_8()

```
void print_bin_8 (
            uint8_t val )
```

Prints the binary representation of an 8-bit unsigned integer.
Affiche la representation binaire (8 bits) d'une valeur.

Parameters

val	The `val` [from 0 to 255]. The function prints the binary representation of this value by iterating over each bit.

13.21.1.7 print_repartition_bit()

```
void print_repartition_bit (
            void  )
```

Affiche une vue textuelle de l'occupation des bits de la trame.
Affiche la repartition des bits utilises dans la trame CAN.

13.21.2 Variable Documentation

13.21.2.1 gbc2

```
tsfp gbc2[9]
```

Espace global de rendu utilise par les fonctions d'impression de repartition.

13.22 tag_manager.c File Reference

```
#include "../../include/sub_include/tag_manager.h"
#include "../../include/config_value.h"
#include "../../include/gestion_tag.h"
#include "../../include/sub_include/print_val.h"
#include <stdio.h>
#include <string.h>
#include <stdint.h>
```

Functions

- void init_tag_manager (void)

  Initialise/reinitialise le gestionnaire de tags.
- int get_first_tags_vide (void)

  [INTERNE] Retourne l'index du premier slot tag libre.
- void set_name_tag (const int n_tag, const char ∗tag_name)

  [INTERNE] Enregistre un nom de tag dans un slot.
- void general_set_tag (const int n_tag, const char ∗tag_name, uint8_t n_bits, int8_t byte_idx_a)

  [INTERNE] Initialise les champs communs d'un tag.
- void set_new_tag_9_to_16 (const char ∗tag_name, const int8_t byte_idx_a, const int8_t byte_idx_b, uint8_t n_bits_total)

  [INTERNE] Cree un tag avec repartition potentielle sur 2 bytes.
- CAN_TG_STATUE find_byte_with_enoung_space (uint8_t siz, int8_t ∗retur_var)

  Trouve le premier byte ayant assez de place libre.
- CAN_TG_STATUE find_byte2_with_enoung_space (uint8_t siz_total, int8_t posi_premier_bit, int8_t ∗retur↩ _var)

  Variante de recherche en excluant le byte deja utilise par la partie A.
- CAN_TG_STATUE set_tag (const char ∗tag_name, uint8_t siz)

  Declare un nouveau tag et reserve son espace dans TxData.
- const TagDef ∗ get_tag_def (const char ∗tag_name)

  Recherche un tag actif par son nom.

Variables

- TagDef g_tags [MAX_TAGS]
- uint8_t g_tag_count = 0
- uint8_t g_bytes_used [N_BYTES] = {0}

13.22.1 Function Documentation

13.22.1.1 find_byte2_with_enoung_space()

```
CAN_TG_STATUE find_byte2_with_enoung_space (
            uint8_t siz_total,
            int8_t posi_premier_bit,
            int8_t * retur_var )
```

Variante de recherche en excluant le byte deja utilise par la partie A.

Parameters

siz_total	Taille totale (9..16).
posi_premier_bit	Byte deja retenu pour la partie A.
retur_var	Sortie: index du byte pour la partie B.

Returns

CAN_TG_SUCCESS ou CAN_TG_ERROR_NOT_ENOUNG_SPACE_.

13.22.1.2 find_byte_with_enoung_space()

```
CAN_TG_STATUE find_byte_with_enoung_space (
            uint8_t siz,
            int8_t * retur_var )
```

Trouve le premier byte ayant assez de place libre.

Parameters

siz	Taille demandee (en bits).
retur_var	Sortie: index du byte selectionne.

Returns

CAN_TG_SUCCESS ou CAN_TG_ERROR_NOT_ENOUNG_SPACE_.

13.22.1.3 general_set_tag()

```
void general_set_tag (
            const int n_tag,
            const char * tag_name,
            uint8_t n_bits,
            int8_t byte_idx_a )
```

[INTERNE] Initialise les champs communs d'un tag.

Parameters

n_tag	Index du slot tag cible.
tag_name	Nom logique a enregistrer.
n_bits	Taille de la valeur en bits.

Parameters

byte_idx↩ _a	Byte principal de stockage (partie A).

13.22.1.4 get_first_tags_vide()

```
int get_first_tags_vide (
            void  )
```

[INTERNE] Retourne l'index du premier slot tag libre.
Un slot est considere libre si son nom est NULL ou vide ('\0').

Returns

Index du slot libre, ou CAN_TG_ERROR_TOO_MAY_TAGS si aucun slot disponible.

13.22.1.5 get_tag_def()

```
const TagDef * get_tag_def (
            const char * tag_name )
```

Recherche un tag actif par son nom.
Recherche un tag et retourne sa definition.

Parameters

tag_name	Nom du tag recherche.

Returns

Pointeur sur TagDef si trouve, sinon NULL.

13.22.1.6 init_tag_manager()

```
void init_tag_manager (
            void  )
```

Initialise/reinitialise le gestionnaire de tags.

Note

Apres cet appel, l'ensemble des tags doit etre redefini via `set_tag`.

13.22.1.7 set_name_tag()

```
void set_name_tag (
            const int n_tag,
            const char * tag_name )
```

[INTERNE] Enregistre un nom de tag dans un slot.

Warning

Les verifications d'entree sont censees etre faites en amont.

Parameters

n_tag	Index du slot (0..MAX_TAGS-1).
tag_name	Nom logique du tag.

13.22.1.8 set_new_tag_9_to_16()

```
void set_new_tag_9_to_16 (
            const char * tag_name,
            const int8_t byte_idx_a,
            const int8_t byte_idx_b,
            uint8_t n_bits_total )
```

[INTERNE] Cree un tag avec repartition potentielle sur 2 bytes.

Parameters

tag_name	Nom logique du tag.
byte_idx↩ _a	Byte de la partie A.
byte_idx↩ _b	Byte de la partie B.
n_bits_total	Taille totale en bits.

13.22.1.9 set_tag()

```
CAN_TG_STATUE set_tag (
            const char * tag_name,
            uint8_t siz )
```

Declare un nouveau tag et reserve son espace dans TxData.
créé un nouveau tag.

Parameters

tag_name	Nom logique du tag.
siz	Taille de la valeur en bits (implementation actuelle: 1..16).

Returns

Code de succes ou code d'erreur CAN_TG_STATUE.

13.22.2 Variable Documentation

13.22.2.1 g_bytes_used

```
uint8_t g_bytes_used[N_BYTES] = {0}
```

Occupation courante en bits pour chaque byte du payload.

13.22.2.2 g_tag_count

```
uint8_t g_tag_count = 0
```

Nombre de tags actifs (max: MAX_TAGS).

13.22.2.3 g_tags

```
TagDef g_tags[MAX_TAGS]
```

Table des tags (actifs et inactifs).

13.23 TAGCAN.h File Reference

```
#include "include/config_value.h"
#include "include/gestion_tag.h"
#include "include/sub_include/data_container.h"
```

```
#include "include/sub_include/tag_manager.h"
```

13.24 test_print_repartition_bit.c File Reference

```
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include "../../include/gestion_tag.h"
#include "../../include/sub_include/data_container.h"
#include "../../include/sub_include/print_val.h"
```

Functions

- int main__ (void)

13.24.1 Function Documentation

13.24.1.1 main__()

```
int main__ (
            void  )
```

13.25 test_safty_func.c File Reference

13.26 test_safty_func.h File Reference

13.27 test_send_receve.c File Reference

13.28 test_send_receve.h File Reference

13.29 test_set_tag.c File Reference

13.30 test_set_tag.h File Reference

13.31 test_utils.c File Reference

```
#include "test_utils.h"
#include <stdbool.h>
#include <stdio.h>
```

Functions

- void TEST_ (const char *name, const bool condition)
- void TEST_EQ_INT_ (const char *name, const int val_1, const int val_2)

Variables

- int total_tests
- int passed_tests

13.31.1 Function Documentation

13.31.1.1 TEST_()

```
void TEST_ (
            const char * name,
            const bool condition )
```

13.31.1.2 TEST_EQ_INT_()

```
void TEST_EQ_INT_ (
            const char * name,
            const int val_1,
            const int val_2 )
```

13.31.2 Variable Documentation

13.31.2.1 passed_tests

```
int passed_tests [extern]
```
le nombre total de test réussi

13.31.2.2 total_tests

```
int total_tests [extern]
```
le nombre total de test effectué

13.32 test_utils.h File Reference

```
#include <stdio.h>
#include <stdbool.h>
```

Functions

- void TEST (const char *name, const bool condition)

  Affiche un resultat de test avec format uniforme.
- void TEST_EQ_INT (const char *name, const int val_1, const int val_2)

  Affiche un test d'egalite entre deux entiers.

Variables

- int total_tests
- int passed_tests

13.32.1 Function Documentation

13.32.1.1 TEST()

```
void TEST (
            const char * name,
            const bool condition )
```
Affiche un resultat de test avec format uniforme.

Parameters

name	Le nom du test (peut aussi servir de description courte).
condition	Resultat du test (true si reussi, false sinon).

13.32.1.2 TEST_EQ_INT()

```
void TEST_EQ_INT (
            const char * name,
            const int val_1,
            const int val_2 )
```

Affiche un test d'egalite entre deux entiers.

Parameters

name	Meme usage que TEST.
val_1	Premiere valeur a comparer.
val_2	Deuxieme valeur a comparer.

13.32.2 Variable Documentation

13.32.2.1 passed_tests

```
int passed_tests [extern]
```

le nombre total de test réussi

13.32.2.2 total_tests

```
int total_tests [extern]
```

le nombre total de test effectué

13.33 version.h File Reference

Macros

- #define RC_FILEVERSION 9,0,24,114
- #define RC_FILEVERSION_STRING "9, 0, 24, 114\0"

13.33.1 Macro Definition Documentation

13.33.1.1 RC_FILEVERSION

```
#define RC_FILEVERSION 9,0,24,114
```

13.33.1.2 RC_FILEVERSION_STRING

```
#define RC_FILEVERSION_STRING "9, 0, 24, 114\0"
```

Index

01 - Initialisation de TAGCAN, 7
01_INITIALISATION.md, 50
02 - Gestion des Tags, 11
02_GESTION_TAGS.md, 50
03 - Manipulation du Buffer de Données CAN, 15
03_DONNEES_CAN.md, 50
04 - Filtrage CAN, 21
04_FILTRAGE_CAN.md, 50
05 - Exemples Complets, 27
05_EXEMPLES_COMPLETS.md, 50

API Reference (Etat actuel), 3
API_REFERENCE.md, 45

bi
    tsfp, 44
bit_pos_a
    TagDef, 43
bit_pos_b
    TagDef, 43
byte_idx_a
    TagDef, 43
byte_idx_b
    TagDef, 43

CAN_change_val
    com_utils.c, 46
CAN_init_TAGCAN
    com_utils.c, 46
CAN_MODE
    config_value.h, 52
CAN_send_msg
    com_utils.c, 46
CAN_setup_DF
    com_utils.c, 46
CAN_setup_Filter
    com_utils.c, 46
CAN_setup_TAG
    com_utils.c, 46
can_simulate_send_receive
    data_container.c, 64
    data_container.h, 55
CAN_TG_ERROR_INTERNAL_CORUPTION
    config_value.c, 60
    config_value.h, 52
CAN_TG_ERROR_INTERNAL_CORUPTION_
    config_value.h, 52
CAN_TG_ERROR_NAME_ALREADY_USE
    config_value.c, 60
    config_value.h, 52
CAN_TG_ERROR_NAME_ALREADY_USE_
    config_value.h, 52
CAN_TG_ERROR_NOT_ENOUNG_SPACE
    config_value.c, 60
    config_value.h, 52
CAN_TG_ERROR_NOT_ENOUNG_SPACE_
    config_value.h, 52
CAN_TG_ERROR_NULL_POINTOR
    config_value.c, 61
    config_value.h, 52
CAN_TG_ERROR_NULL_POINTOR_
    config_value.h, 52
CAN_TG_ERROR_SIZE_VALUE_INVALIDE
    config_value.c, 61
    config_value.h, 53
CAN_TG_ERROR_SIZE_VALUE_INVALIDE_
    config_value.h, 52
CAN_TG_ERROR_SUBDIVISION_FAIL
    config_value.c, 61
    config_value.h, 53
CAN_TG_ERROR_SUBDIVISION_FAIL_
    config_value.h, 52
CAN_TG_ERROR_TAG_NAME_INVALIDE
    config_value.c, 61
    config_value.h, 53
CAN_TG_ERROR_TAG_NAME_INVALIDE_
    config_value.h, 52
CAN_TG_ERROR_TAG_NAME_TO_BIGS
    config_value.c, 61
    config_value.h, 53
CAN_TG_ERROR_TAG_NAME_TO_BIGS_
    config_value.h, 52
CAN_TG_ERROR_TAG_NOT_FOUND
    config_value.c, 61
    config_value.h, 53
CAN_TG_ERROR_TAG_NOT_FOUND_
    config_value.h, 52
CAN_TG_ERROR_TOO_MAY_TAGS
    config_value.c, 61
    config_value.h, 53
CAN_TG_ERROR_TOO_MAY_TAGS_
    config_value.h, 52
CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER
    config_value.c, 61
    config_value.h, 53
CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER_
    config_value.h, 52
CAN_TG_ERROR_VALUE_TO_BIG
    config_value.h, 52

CAN_TG_STATUE
config_value.h, 52
CAN_TG_SUCCESS
config_value.h, 52
CAN_utils.c, 60
com_utils.c, 45
CAN_change_val, 46
CAN_init_TAGCAN, 46
CAN_send_msg, 46
CAN_setup_DF, 46
CAN_setup_Filter, 46
CAN_setup_TAG, 46
datacheck, 47
HAL_CAN_RxFifo0MsgPendingCallback, 46
init_com, 46
main_loop, 46
out_Rx, 47
RxData, 47
RxData_moteur, 47
RxHeader, 47
TxData_moteur, 47
TxHeader, 47
TxMailbox, 47
update_all_val_receve, 47
update_all_val_send, 47
update_TagData, 47
update_val, 47
val_AB, 47
val_CI, 48
val_EDC, 48
val_HB, 48
val_N2ODVS, 48
val_N2OIV, 48
val_N2OMV, 48
val_N2OPS, 48
val_SEQ, 48
val_TIME, 48
val_TLC, 48
com_utils.h, 48
hcan, 49
init_com, 49
RxData, 49
RxHeader, 49
TxData, 49
TxHeader, 49
TxMailbox, 49
val_AB, 49
val_CI, 50
val_EDC, 50
val_HB, 50
val_N2ODVS, 50
val_N2OIV, 50
val_N2OMV, 50
val_N2OPS, 50
val_SEQ, 50
val_TIME, 50
val_TLC, 50
config_value.c, 60
CAN_TG_ERROR_INTERNAL_CORUPTION, 60
CAN_TG_ERROR_NAME_ALREADY_USE, 60
CAN_TG_ERROR_NOT_ENOUNG_SPACE, 60
CAN_TG_ERROR_NULL_POINTOR, 61
CAN_TG_ERROR_SIZE_VALUE_INVALIDE, 61
CAN_TG_ERROR_SUBDIVISION_FAIL, 61
CAN_TG_ERROR_TAG_NAME_INVALIDE, 61
CAN_TG_ERROR_TAG_NAME_TO_BIGS, 61
CAN_TG_ERROR_TAG_NOT_FOUND, 61
CAN_TG_ERROR_TOO_MAY_TAGS, 61
CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER, 61
SUCCES, 61
SUCCES_TO_SET, 61
config_value.h, 50
CAN_MODE, 52
CAN_TG_ERROR_INTERNAL_CORUPTION, 52
CAN_TG_ERROR_INTERNAL_CORUPTION_, 52
CAN_TG_ERROR_NAME_ALREADY_USE, 52
CAN_TG_ERROR_NAME_ALREADY_USE_, 52
CAN_TG_ERROR_NOT_ENOUNG_SPACE, 52
CAN_TG_ERROR_NOT_ENOUNG_SPACE_, 52
CAN_TG_ERROR_NULL_POINTOR, 52
CAN_TG_ERROR_NULL_POINTOR_, 52
CAN_TG_ERROR_SIZE_VALUE_INVALIDE, 53
CAN_TG_ERROR_SIZE_VALUE_INVALIDE_, 52
CAN_TG_ERROR_SUBDIVISION_FAIL, 53
CAN_TG_ERROR_SUBDIVISION_FAIL_, 52
CAN_TG_ERROR_TAG_NAME_INVALIDE, 53
CAN_TG_ERROR_TAG_NAME_INVALIDE_, 52
CAN_TG_ERROR_TAG_NAME_TO_BIGS, 53
CAN_TG_ERROR_TAG_NAME_TO_BIGS_, 52
CAN_TG_ERROR_TAG_NOT_FOUND, 53
CAN_TG_ERROR_TAG_NOT_FOUND_, 52
CAN_TG_ERROR_TOO_MAY_TAGS, 53
CAN_TG_ERROR_TOO_MAY_TAGS_, 52
CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER, 53
CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER_, 52
CAN_TG_ERROR_VALUE_TO_BIG, 52
CAN_TG_STATUE, 52
CAN_TG_SUCCESS, 52
MAX_N_GT, 51
MAX_TAG_NAME_LEN, 51
MAX_TAGS, 51
N_BYTES, 51
RECEVE_ONLY, 52
SEND_AND_RECEVE, 52
SEND_ONLY, 52
SUCCES, 53
SUCCES_TO_SET, 53
convert_8_to_16bit
data_container.c, 64

data_container.c, 63
can_simulate_send_receive, 64
convert_8_to_16bit, 64
get_value, 64

init_data_container, 65
prep_mask, 65
set_val_TxData, 65
set_value, 65
TxData, 67
data_container.h, 54
can_simulate_send_receive, 55
get_value, 55
init_data_container, 56
prep_mask, 56
set_val_TxData, 56
set_value, 56
TxData, 57
datacheck
com_utils.c, 47
Documentation TAGCAN, 1

find_byte2_with_enoung_space
tag_manager.c, 70
find_byte_with_enoung_space
tag_manager.c, 70

g_bytes_used
tag_manager.c, 72
tag_manager.h, 60
g_tag_count
tag_manager.c, 72
tag_manager.h, 60
g_tags
tag_manager.c, 72
tag_manager.h, 60
gbc2
print_val.c, 69
print_val.h, 59
general_set_tag
tag_manager.c, 70
gestion_tag.h, 53
init_tag_manager, 54
set_tag, 54
get_first_tags_vide
tag_manager.c, 71
get_tag_def
tag_manager.c, 71
tag_manager.h, 59
get_value
data_container.c, 64
data_container.h, 55
Guide d'Utilisation de TAGCAN, 35

HAL_CAN_RxFifo0MsgPendingCallback
com_utils.c, 46
hcan
com_utils.h, 49

init_com
com_utils.c, 46
com_utils.h, 49
init_data_container
data_container.c, 65
data_container.h, 56
init_tag_manager
gestion_tag.h, 54
tag_manager.c, 71

main
main.c, 62
main.c, 62
main, 62
passed_tests, 63
run_all_tests, 62
somme_other_test, 62
TEST, 62
TEST_EQ_INT, 63
test_print, 63
total_tests, 63
main__
test_print_repartition_bit.c, 73
main_loop
com_utils.c, 46
MAX_N_GT
config_value.h, 51
MAX_TAG_NAME_LEN
config_value.h, 51
MAX_TAGS
config_value.h, 51
modif_vt
print_val.c, 67

n_bits
TagDef, 43
N_BYTES
config_value.h, 51
name
TagDef, 43

out_Rx
com_utils.c, 47

passed_tests
main.c, 63
test_utils.c, 74
test_utils.h, 75
prep_mask
data_container.c, 65
data_container.h, 56
print_all_data
print_val.c, 68
print_val.h, 57
print_all_data_bin
print_val.c, 68
print_val.h, 57
print_all_tag_set
print_val.c, 68
print_val.h, 58
print_all_with_title
print_val.c, 68
print_val.h, 58
print_bin_8

print_val.c, 68
print_val.h, 58
print_repartition_bit
print_val.c, 69
print_val.h, 58
print_val.c, 67
gbc2, 69
modif_vt, 67
print_all_data, 68
print_all_data_bin, 68
print_all_tag_set, 68
print_all_with_title, 68
print_bin_8, 68
print_repartition_bit, 69
print_val.h, 57
gbc2, 59
print_all_data, 57
print_all_data_bin, 57
print_all_tag_set, 58
print_all_with_title, 58
print_bin_8, 58
print_repartition_bit, 58

RC_FILEVERSION
version.h, 75
RC_FILEVERSION_STRING
version.h, 75
README.md, 45
RECEVE_ONLY
config_value.h, 52
run_all_tests
main.c, 62
RxData
com_utils.c, 47
com_utils.h, 49
RxData_moteur
com_utils.c, 47
RxHeader
com_utils.c, 47
com_utils.h, 49

SEND_AND_RECEVE
config_value.h, 52
SEND_ONLY
config_value.h, 52
set_name_tag
tag_manager.c, 71
set_new_tag_9_to_16
tag_manager.c, 72
set_tag
gestion_tag.h, 54
tag_manager.c, 72
set_val_TxData
data_container.c, 65
data_container.h, 56
set_value
data_container.c, 65
data_container.h, 56
somme_other_test
main.c, 62
SUCCES
config_value.c, 61
config_value.h, 53
SUCCES_TO_SET
config_value.c, 61
config_value.h, 53

tag_manager.c, 69
find_byte2_with_enoung_space, 70
find_byte_with_enoung_space, 70
g_bytes_used, 72
g_tag_count, 72
g_tags, 72
general_set_tag, 70
get_first_tags_vide, 71
get_tag_def, 71
init_tag_manager, 71
set_name_tag, 71
set_new_tag_9_to_16, 72
set_tag, 72
tag_manager.h, 59
g_bytes_used, 60
g_tag_count, 60
g_tags, 60
get_tag_def, 59
TAGCAN, 37
TAGCAN.h, 72
TagDef, 43
bit_pos_a, 43
bit_pos_b, 43
byte_idx_a, 43
byte_idx_b, 43
n_bits, 43
name, 43
TEST
main.c, 62
test_utils.h, 74
TEST_
test_utils.c, 74
TEST_EQ_INT
main.c, 63
test_utils.h, 74
TEST_EQ_INT_
test_utils.c, 74
test_print
main.c, 63
test_print_repartition_bit.c, 73
main__, 73
test_safty_func.c, 73
test_safty_func.h, 73
test_send_receve.c, 73
test_send_receve.h, 73
test_set_tag.c, 73
test_set_tag.h, 73
test_utils.c, 73
passed_tests, 74
TEST_, 74
TEST_EQ_INT_, 74

total_tests, 74
test_utils.h, 74
passed_tests, 75
TEST, 74
TEST_EQ_INT, 74
total_tests, 75
total_tests
main.c, 63
test_utils.c, 74
test_utils.h, 75
tsfp, 44
bi, 44
TxData
com_utils.h, 49
data_container.c, 67
data_container.h, 57
TxData_moteur
com_utils.c, 47
TxHeader
com_utils.c, 47
com_utils.h, 49
TxMailbox
com_utils.c, 47
com_utils.h, 49

update_all_val_receve
com_utils.c, 47
update_all_val_send
com_utils.c, 47
update_TagData
com_utils.c, 47
update_val
com_utils.c, 47

val_AB
com_utils.c, 47
com_utils.h, 49
val_CI
com_utils.c, 48
com_utils.h, 50
val_EDC
com_utils.c, 48
com_utils.h, 50
val_HB
com_utils.c, 48
com_utils.h, 50
val_N2ODVS
com_utils.c, 48
com_utils.h, 50
val_N2OIV
com_utils.c, 48
com_utils.h, 50
val_N2OMV
com_utils.c, 48
com_utils.h, 50
val_N2OPS
com_utils.c, 48
com_utils.h, 50
val_SEQ
com_utils.c, 48
com_utils.h, 50
val_TIME
com_utils.c, 48
com_utils.h, 50
val_TLC
com_utils.c, 48
com_utils.h, 50
version.h, 75
RC_FILEVERSION, 75
RC_FILEVERSION_STRING, 75